FRAME STUDIO: AN AI-POWERED PLATFORM FOR DYNAMIC MULTIMEDIA GENERATION



Bachelor of Science (Software Engineering)

Session (2022-2026)

Submitted By:

Muhammad Farooq (2022F-MULBSSWE-014)

SUPERVISOR

Dr. Muhammad Saleem

Assistant Professor

Department of Software Engineering

SCHOOL OF SOFTWARE ENGINEERING

MINHAJ UNIVERSITY, LAHORE

FRAME STUDIO: AN AI-POWERED PLATFORM FOR DYNAMIC MULTIMEDIA GENERATION

Project

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science (Software Engineering)

at the Minhaj University, Lahore

By

Muhammad Farooq Reg. No. 2022F-MULBSSWE-014

Dr. Muhammad Saleem	Mr. Imran Akhtar
Assistant Professor School of Software Engineering	Head of Department School of Software Engineering

DECLARATION

It is declared that this is an original piece of my own work, except where otherwise acknowledged in text and references. This work has not been submitted in any form for another degree or diploma at any university or other institution for tertiary education and shall not be submitted by me in future for obtaining any degree from this or any other University or Institution.

Muhammad Farooq

Signature: _____

Reg. No. 2022F-MULBSSWE-014

DEDICATION

To

My Parents

For their endless prayers, patience and sacrifices, for the quiet certainty that I could do this even on the days I doubted it myself, and for everything they gave up so that I would never have to choose between an education and anything else.

My Teachers

For the guidance, patience and knowledge that shaped the way I think about engineering, and for asking the questions in class that I only understood the value of once I was debugging this project late into the night.

My Friends and Classmates

For testing early, broken versions of this platform without complaint, and for the honest feedback that made the finished system better than anything I would have built working entirely alone.

CERTIFICATE OF APPROVAL

It is certified that the project titled "FRAME STUDIO: AN AI-POWERED PLATFORM FOR DYNAMIC MULTIMEDIA GENERATION", carried out by Muhammad Farooq (2022F-MULBSSWE-014) under the supervision of Dr. Muhammad Saleem, Minhaj University, Lahore, meets the required scope and quality for the Final Year Project of the Bachelor of Science in Software Engineering. The undersigned recommend its acceptance for the degree.

Supervisor:

Dr. Muhammad Saleem
School of Software Engineering

Internal Examiner 1:

School of Software Engineering

Internal Examiner 2:

School of Software Engineering

FYP Coordinator:

Mr. Muhammad Kamran
School of Software Engineering

Head of Department:

Mr. Imran Akhtar
School of Software Engineering

Dean:

Prof. Dr. Abdul Aziz
Dean Faculty of CS & IT

ACKNOWLEDGEMENT

All praise is due to Allah Almighty for granting me the strength, patience and opportunity to complete this project. I would like to express my sincere gratitude to my supervisor, Dr. Muhammad Saleem, for the guidance, feedback and encouragement throughout the development of this project. His advice was instrumental whenever the project's technical direction needed to be reconsidered.

I am deeply grateful to my parents for their unconditional support, patience and prayers throughout my academic journey. Their encouragement gave me the confidence to take on a project of this scale and see it through to completion.

I would also like to thank my teachers at the Department of Software Engineering, Minhaj University Lahore, for the knowledge and skills they imparted over the course of my degree, much of which was applied directly in building this system. Finally, I thank my friends and classmates who tested early versions of the platform and offered honest feedback that helped improve it.

ABSTRACT

FRAME Studio is a full-stack web platform for generating and editing short documentary-style videos, images, and image analyses using AI. A user supplies a topic and a target length between one and twenty minutes; the system writes a narration script, generates or sources matching visuals, synthesises voice narration, and composes the result into a finished video with transitions, background music, and burned-in subtitles using ffmpeg. Beyond one-shot generation, the platform includes a scene-level Video Editor that lets a user re-narrate individual scenes, swap images, apply visual effects, and re-render only what changed rather than the whole video; an Image Studio for standalone AI image generation; an Image Analyzer that returns labels, a natural-language description, and ready-to-use follow-up prompts for a photo, exportable as a PDF report; and an agentic chat mode in which a large language model can call the same generation tools on the user's behalf after clarifying details such as length and visual style.

The system is built as a Node.js/Express backend with MongoDB for persistence and a React/Material-UI frontend. Because every generation step depends on a third-party API, the backend is built around multi-provider fallback chains rather than a single vendor: text generation tries DeepSeek, then Groq, then Gemini; text-to-speech tries ElevenLabs' natural voices and falls back to Google TTS when a key is missing or a quota is exhausted; image generation tries Flux (via Hugging Face), then Gemini, then other configured providers. Long-running work such as video rendering runs through a lease-based background job queue so a server restart never silently duplicates or loses a render. Security work included moving all generated media behind authenticated, short-lived tokens instead of open static file mounts, closing path-traversal and SSRF gaps in file handling, adding JWT access/refresh rotation with session revocation, and adding per-user daily quotas and rate limits so an open signup flow cannot be used to run up an unbounded third-party API bill.

The backend carries 248 automated tests across 21 test files covering narration generation, video length scaling, ffmpeg filter correctness, image analysis, job payload validation, security middleware, rate limiting, and pagination. Frontend automated testing and coverage of some services (authentication, TTS, the chat agent) were not completed within the project timeline and are documented as a known limitation and direction for future work. This document describes the requirements, design, implementation, and testing of the system, and reflects honestly on what worked, what did not, and what remains to be done.

LIST OF FIGURES

Figure 1: Use Case Diagram - FRAME Studio

Figure 2: System Architecture Diagram

Figure 2b: System Architecture Diagram (reference rendering)

Figure 3: Entity Relationship Diagram (ERD)

Figure 4: Data Flow Diagram - Level 0

Figure 5: Data Flow Diagram - Level 1

Figure 6: Class Diagram

Figure 7: Activity Diagram - Generate Video

Figure 8: Sequence Diagram - Generate Video

Figure 9: Collaboration Diagram - Generate Video

Figure 10: State Transition Diagram - Background Job Lifecycle

Figure 11: Component Diagram

Figure 12: Deployment Diagram

Figure 13: Work Breakdown Structure

Figure 14: Gantt Chart

LIST OF TABLES

Table 1: FR-01 Register Account

Table 2: FR-02 Login

Table 3: FR-03 Generate Video

Table 4: FR-04 Edit Video - Scenes

Table 5: FR-05 Edit Video - Apply Effects

Table 6: FR-06 Generate Image

Table 7: FR-07 Analyze Image

Table 8: FR-08 Chat with AI Assistant

Table 9: FR-09 View History / Dashboard

Table 10: FR-10 Delete Item

Table 11: FR-11 Download Item

Table 12: FR-12 Manage Queue (Background Jobs)

Table 13: FR-13 Change TTS Voice

Table 14: FR-14 Change Password

Table 15: FR-15 Sign Out Everywhere

Table 16: UC-01 Register Account

Table 17: UC-02 Login

Table 18: UC-03 Generate Video

Table 19: UC-04 Edit Video (Scenes / Effects)

Table 20: UC-05 Generate Image

Table 21: UC-06 Analyze Image

Table 22: UC-07 Chat with AI Assistant

Table 23: UC-08 Manage History (View / Delete / Download)

Table 24: UC-09 Change Password

Table 25: UC-10 Sign Out Everywhere

Table 26: UC-11 Monitor Job Queue

Table 27: TC-01 Successful login with correct email and password

Table 28: TC-02 Failed login with wrong password

Table 29: TC-03 Generate video with a valid prompt

Table 30: TC-04 Generate video with an empty prompt (validation)

Table 31: TC-05 Apply a visual effect to an existing video

Table 32: TC-06 Delete a video the user owns

Table 33: TC-07 Change password with the correct current password

Table 34: TC-08 Reject a password change when the new password equals the current one

Table 35: TC-09 Sign out everywhere revokes sessions on other devices

TABLE OF CONTENTS

Declaration..... 3

Dedication..... 4

Certificate of Approval..... 5

Acknowledgement..... 6

Abstract..... 7

List of Figures..... 8

List of Tables..... 9

CHAPTER 1: INTRODUCTION TO THE PROBLEM..... 14

1.6 Project Plan..... 18

CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION..... 20

CHAPTER 3: USE CASE ANALYSIS..... 33

Use Case Diagram Description..... 34

CHAPTER 4: DESIGN..... 45

Architecture Diagram..... 46

Entity Relationship Diagram (ERD)..... 48

DFD Level 0 (Context Diagram)..... 49

DFD Level 1..... 50

Class Diagram..... 51

Activity Diagram..... 53

Sequence Diagram..... 55

Collaboration Diagram..... 55

State Transition Diagram..... 56

Component Diagram..... 57

Deployment Diagram..... 59

CHAPTER 5: TESTING..... 61

CHAPTER 6: TOOLS AND TECHNIQUES..... 72

Languages..... 73

Frameworks and Libraries..... 73

Database..... 73

Media Processing..... 73

AI / LLM Providers..... 74

Text-to-Speech..... 74

Image Generation..... 75

Computer Vision..... 75

Stock Photos..... 75

Authentication and Security..... 75

DevOps..... 76

Testing..... 76

Version Control..... 76

CHAPTER 7: SUMMARY AND CONCLUSION..... 77

CHAPTER 8: USER MANUAL..... 80

CHAPTER 9: LESSONS LEARNT AND FUTURE WORK..... 92

REFERENCES..... 97

APPENDIX A: ENVIRONMENT SETUP AND INSTALLATION GUIDE..... 100

A.1 Overview..... 101

A.2 Prerequisites..... 101

A.3 Option A — Local Node (development)..... 101

A.4 Option B — Docker Compose (whole stack)..... 102

A.5 Common operational commands..... 103

A.6 Known setup caveats..... 103

A.7 Key backend API routes referenced in this document..... 103

A.8 Option C — Live deployment (Vercel + Railway)..... 104

CHAPTER 1:
INTRODUCTION TO THE PROBLEM

1.1 Introduction

Producing a short narrated video used to require writing a script, sourcing or drawing images, recording or synthesising a voiceover, and cutting the pieces together in a video editor. Each step is its own skill and its own piece of software. Over the last few years a number of AI tools have appeared that try to automate one or more of these steps individually — text generation, image generation, text-to-speech — but a person who wants to go from "an idea for a video" to "a finished video with narration, visuals, and subtitles" still has to operate several separate tools and manually stitch their outputs together.

FRAME Studio is a single platform that does this whole pipeline in one place: a user types a topic, picks a length and a visual style, and the system writes the narration, sources or generates the images, synthesises the voice, and renders the final video. It also goes one step further than most "generate and done" tools by giving the user a proper editor afterwards — the ability to open a generated video, change a scene's narration or image, apply a visual effect, and re-render only the parts that changed.

1.2 Purpose

The purpose of this project is to build and evaluate a working content-generation platform that turns a short text prompt into a complete narrated video, image, or image analysis, while remaining editable and reliable enough for a real user rather than a one-off demo. The system needed to handle failure gracefully — third-party AI APIs are unreliable, rate-limited, and occasionally deprecate models without notice — and it needed to be usable by more than one person at a time without one user's activity affecting another's, which meant real authentication, per-user data isolation, and cost controls.

1.3 Objective

The concrete objectives of the project were:

- To build an end-to-end pipeline that converts a text prompt into a narrated video: script generation, image sourcing (AI-generated or stock), text-to-speech narration, and ffmpeg-based video composition with transitions, background music, and burned-in subtitles.
- To support a configurable video length from one to twenty minutes, with the script's depth and detail scaling appropriately rather than simply repeating filler content.
- To build a Video Editor that operates at the scene level, so a user can re-narrate one scene, swap one image, or apply an effect without regenerating the entire video from scratch.

- To build a standalone Image Studio for AI image generation and an Image Analyzer that returns real labels, a description, and follow-up prompt suggestions for an uploaded photo.
- To build an agentic chat mode where a language model can drive the same generation tools conversationally, asking clarifying questions before acting rather than guessing.
- To make the system resilient to third-party API failure through provider fallback chains, so a single expired API key or exhausted quota does not take a feature down entirely.
- To secure the platform properly: authenticated media delivery, JWT-based sessions with revocation, protection against path traversal and server-side request forgery, and per-user rate limits and daily quotas.
- To keep long-running work (video rendering, image generation) off the request/response cycle through a background job queue, so the UI stays responsive and work survives a page reload.

1.4 Existing Solution

Several categories of existing tools address parts of this problem:

Synthesia and similar avatar-video platforms focus on presenter-style videos with a talking digital avatar reading a script, aimed mainly at corporate training and explainer content. They are strong at lip-synced avatars but are not built for documentary-style narration over generated or stock visuals, and scene-level re-editing after generation is limited.

Pictory and InVideo are aimed at turning existing long-form content (a blog post, a webinar recording) into short social clips, or assembling video from stock footage libraries with templated transitions. Their AI writing and voice options exist but are secondary to their template-and-stock-library workflow, and they do not expose an agentic assistant that can plan a whole generation on the user's behalf.

Generic ChatGPT-based approaches — asking a chatbot to write a script, then pasting that script into a separate TTS tool, then pasting the audio and some stock images into a separate video editor — technically work, but the user does all of the orchestration and file-shuffling by hand. There is no persistence of past work in one place, no unified per-scene editing after the fact, and no single account or dashboard to manage a growing library of generated content.

A recurring limitation across these tools is single-provider dependency: if the underlying TTS engine or image API used by the tool has an outage, hits a quota, or is deprecated, the feature simply stops working for every user until the vendor fixes it or the product team ships a patch.

None of the mainstream tools surfaced expose a visible, automatic fallback to a second or third provider.

1.5 Proposed Solution

FRAME Studio differs from the tools above in three concrete ways, all grounded in what is actually implemented in the codebase rather than in a general aspiration:

First, generation and editing are unified. A video produced by the pipeline is stored with its individual scenes (image, narration, subtitle, duration) as its own array on the Video document, not just as a flattened final file. This is what makes the Video Editor's scene-level re-render possible: changing one scene's text triggers re-narration and re-render of only that scene's audio, while every other scene reuses its already-rendered narration clip, instead of re-running the whole pipeline.

Second, reliability comes from explicit provider fallback chains rather than a single vendor integration. Text generation for scripts and chat tries DeepSeek first, then Groq, then Gemini, in an order that can be reconfigured through environment variables without a code change. Text-to-speech tries an ElevenLabs natural voice first and falls back to Google's gTTS the moment ElevenLabs returns a 401 (bad key) or 429 (quota exhausted), so a video's narration still gets produced instead of failing outright. Image generation tries Flux through Hugging Face Spaces first, then Gemini's image model, with the failure reason from each provider captured and surfaced to the user instead of a generic "something went wrong."

Third, the platform treats cost and abuse as first-class engineering concerns from the start, not an afterthought. Every generation endpoint that spends money against a third-party API sits behind a per-user daily quota and a per-user rate limiter, counted from durable database records rather than an in-memory counter that would reset on every restart. Generated media is served only to authenticated requests through short-lived signed tokens rather than open static file mounts, which also closes off the kind of guessable-URL data leak that affects simpler implementations. These were not requirements handed down by a client; they emerged from actually running the system and discovering, for example, that an unbounded `/downloads` static mount meant anyone who guessed or intercepted a filename could view another user's generated video.

1.6 Project Plan

The project was carried out over the seventh and eighth semesters, from January 2026 through September 2026, following the phased plan summarised below. Work proceeded roughly in the order requirement analysis, system design, backend implementation, frontend implementation, testing and security hardening, deployment, and documentation, though later phases overlapped with earlier ones as features were implemented and tested incrementally rather than in a strict waterfall sequence.

1.6.1 Work Breakdown Structure

The work was broken down into six top-level packages, each grouping the related functional requirements described in Chapter 2: account and authentication, video generation and editing, image generation and analysis, the chat assistant and background job queue, testing and security, and deployment and documentation.

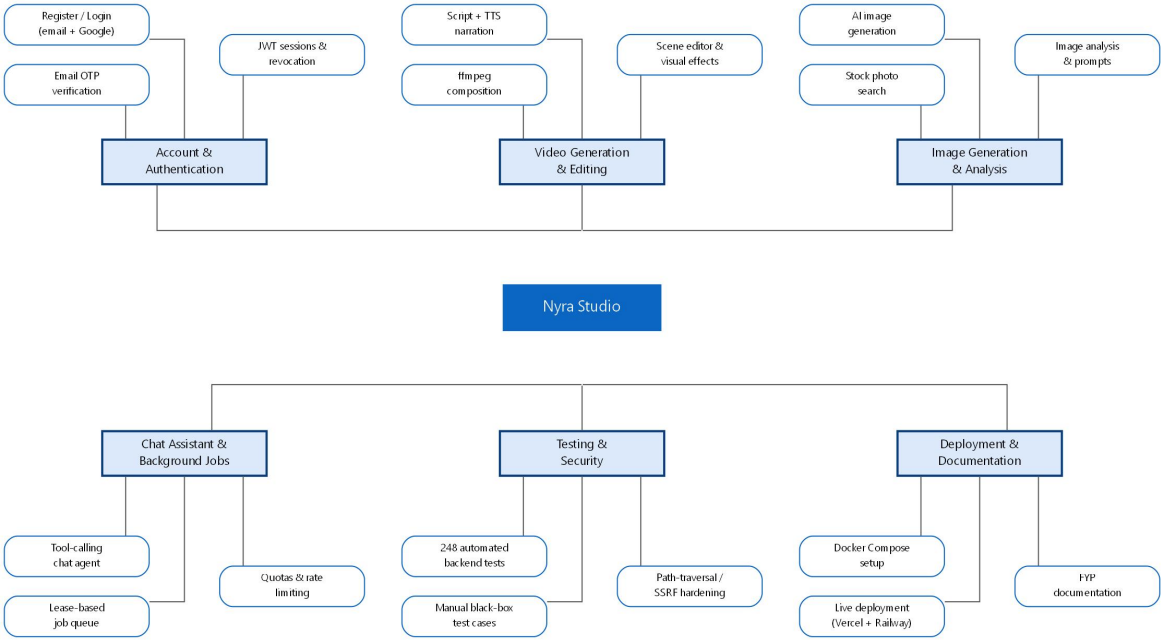


Figure 13: Work Breakdown Structure

1.6.2 Gantt Chart

The timeline below maps each work package onto the months in which it was carried out. Later phases begin before earlier ones fully close because backend and frontend work, and testing alongside it, proceeded feature-by-feature rather than as strictly separate stages.

Task	Jan 2026	Feb 2026	Mar 2026	Apr 2026	May 2026	Jun 2026	Jul 2026	Aug 2026	Sep 2026
FYP proposal & topic approval									
Requirement analysis & SRS									
System design (ERD, class, architecture diagrams)									
Backend implementation (auth, generation pipeline, jobs)									
Frontend implementation (Studio, Editor, Dashboard)									
Testing, security hardening & automated test suite									
Deployment (Docker, Vercel + Railway)									
Documentation & final submission									

Figure 14: Gantt Chart

CHAPTER 2:
SOFTWARE REQUIREMENT SPECIFICATION

2.1 Introduction

2.1.1 Purpose

This chapter specifies the functional and non-functional requirements of FRAME Studio, following the standard functional/non-functional requirement structure described in the software engineering literature [1]. It is written from the actual implemented system (backend routes, controllers, services, and models) rather than as an aspirational document, so every requirement below corresponds to code that exists in the repository at the time of writing.

2.1.2 Scope

The system in scope covers: user registration and authentication (including Google sign-in), AI video generation from a text prompt, scene-level video editing (re-narration, image replacement, visual effects, subtitle editing), standalone AI image generation, image analysis with label detection and prompt suggestions, an agentic chat assistant that can invoke the same generation capabilities, a background job queue for long-running work, and a dashboard/history system for viewing, downloading, and deleting previously generated content. Out of scope: billing/payment processing, multi-language UI, and native mobile applications (see Chapter 9 for future work).

2.1.3 Definitions, Acronyms, and Abbreviations

- API: Application Programming Interface
- JWT: JSON Web Token, used here for stateless session authentication
- TTS: Text-to-Speech
- LLM: Large Language Model (e.g. DeepSeek, Groq-hosted models, Gemini)
- SSE: Server-Sent Events, a one-way HTTP streaming mechanism used for live progress updates
- SSRF: Server-Side Request Forgery, an attack where a server is tricked into making a request to an unintended destination
- ffmpeg: an open-source command-line tool for video/audio processing, used here for video composition and effects
- Job/Queue: a persisted unit of background work (video render, image generation, editor re-render) processed by a worker outside the HTTP request cycle
- Scene: one segment of a generated video, consisting of an image, a narration clip, a subtitle line, and a duration

- Quota: a per-user daily cap on how many generation requests of a given kind (video/image) may be started

2.2 Overall Description

2.2.1 Product Perspective

FRAME Studio is a standalone, self-contained web application (not an add-on to an existing system), composed of a React single-page frontend and an Express.js REST API backend, backed by MongoDB.

System interfaces: The backend exposes a REST API under /api (auth, user, video, chat, jobs, media, and general content routes). Real-time video generation progress is delivered to the frontend over Server-Sent Events rather than polling for the initial synchronous generation path; background jobs (queued video/image/render work) are polled periodically by the frontend against /api/jobs.

User interfaces: A React 18 + Material-UI frontend with dedicated pages for the landing page, sign-in/sign-up, onboarding, the Studio (video generation and library), the Video Editor, the Image Generator, the Image Analyzer, the Chat assistant, the Dashboard (unified history), and Settings.

Hardware interfaces: No special hardware is required beyond a server capable of running Node.js and ffmpeg (video composition is CPU-bound) and a client device with a modern browser. No GPU is required on the server because AI image/text generation is delegated to external provider APIs.

Software interfaces: MongoDB (via Mongoose ODM) for all persistent data; Cloudinary for durable storage and signed delivery of generated images/videos, with local disk (`public/downloads`, `public/videos`, etc.) used only as transient scratch space during generation; ffmpeg/fluent-ffmpeg for video composition, transitions, subtitle burning, and effects; and a set of third-party AI APIs — DeepSeek, Groq, and Google Gemini for text generation; ElevenLabs and Google TTS (gTTS) for narration; Hugging Face-hosted Flux and Gemini for image generation; Google Cloud Vision for supplementary image labelling; and Pexels for stock photography.

Communications interfaces: HTTPS REST over JSON for all API calls; JWT bearer tokens in the Authorization header for authenticated requests; a short-lived, separately-scoped

"media token" appended as a query parameter for `<img>`/`<video>` tags that cannot set custom headers; Server-Sent Events for live generation progress.

2.2.2 Product Functions

The following functional requirements are drawn directly from the implemented controllers and routes.

FR-01

Name	Register Account
Description	Create a new user account with a username, email, and password, and send an OTP email verification code.
Input	username, email, password
Output	pending verification response (email, otpSent flag); on later verification, an access token, a refresh token, and the user profile
Basic Work Flow	Client submits signup form -> AuthController.signup -> AuthService checks for an existing account (without revealing which case applies, to avoid email enumeration) -> a bcrypt-hashed 6-digit OTP is generated and stored with a 10-minute expiry -> OTP emailed via Resend -> user submits the code to /api/auth/verify-email -> account marked emailVerified, JWT pair issued.

FR-02

Name	Login
Description	Authenticate an existing user with email and password (or Google credential) and issue a session.
Input	email, password (or a Google ID token for FR-02b)
Output	access token, refresh token, user profile
Basic Work Flow	Client submits credentials -> AuthController.login -> AuthService.loginService compares the bcrypt hash -> if the account is unverified, a fresh OTP is issued and a 403 EMAIL_NOT_VERIFIED response is returned instead of logging in -> on success, lastLoginAt is updated and a JWT pair (carrying the account's current tokenVersion) is returned.

FR-03

Name	Generate Video
Description	Produce a narrated video from a topic, a length in minutes (1-20), an image source (AI-generated, stock, local upload, or mixed), and a TTS voice choice.
Input	prompt, videoLength, imageSource, ttsProvider, optionally selectedImages
Output	a videoUrl and videoId once complete, or a jobId for background tracking with live progress events
Basic Work Flow	Prompt submitted from the Studio -> a script is generated and split into per-scene narration lines -> for each line, an image is generated or sourced and TTS narration is synthesised -> VideoService composes the segments with ffmpeg (transitions, subtitles, background music) -> the finished file is uploaded to Cloudinary, a Video document is created, and the URL is returned.

FR-04

Name	Edit Video — Scenes
Description	Re-render a saved video from an edited scene list, re-narrating only the scenes whose text changed.
Input	videoId, the full scenes array (image/narration/subtitle/duration per scene), changedSubtitleIndexes, transitionSec, transitionType, ttsProvider
Output	a jobId (queued path) or an immediate videoUrl/videoId (synchronous fallback path)
Basic Work Flow	Editor's "Save Edit" submits the staged scene list -> VideoEditService normalises and validates the payload -> for each changed scene, new narration audio is synthesised; unchanged scenes reuse their existing rendered clip -> VideoService.renderFromScenes recomposes the video -> the new render is uploaded and filed as a new Video record.

FR-05

Name	Edit Video — Apply Effects
Description	Apply one or more visual effects (e.g. color grading, film grain, cinematic

	zoom) to an existing video.
Input	videoId, effects (array, max 20), optional prompt/imageSource/ttsProvider
Output	a jobId (queued) or an immediate videoUrl/videoId
Basic Work Flow	Editor's Effects tab submits selected effects -> VideoEffectsService validates the payload and resolves the source video path (with SSRF/path-containment checks if the source is a remote URL) -> VideoEditorService builds and runs the corresponding ffmpeg filter chain -> result is uploaded and filed as a new Video record.

FR-06

Name	Generate Image
Description	Generate a single AI image from a text prompt and save it to the user's image library.
Input	prompt
Output	imageUrl, assetId, imageId, persisted flag
Basic Work Flow	Prompt submitted from the Image Studio -> ImageGeneratorService tries the configured provider chain (Flux, then Gemini, then others) -> on success the file is uploaded to Cloudinary, an Asset and an Image document are created -> the delivery URL is returned; if storage fails, the image is still returned but flagged persisted:false.

FR-07

Name	Analyze Image
Description	Analyze an uploaded photo and return labels, a natural-language description, and 2-3 suggested prompts for recreating or riffing on it.
Input	an uploaded image file (multipart)
Output	labels, description, suggestedPrompts, analysisId, imageUrl
Basic Work Flow	File uploaded via the Image Analyzer -> ImageAnalysisService sends the image to a vision-capable LLM with a structured JSON prompt -> Google Cloud Vision labels are merged in if configured -> result is uploaded to

	Cloudinary and persisted as an Analysis document -> the same function backs both the page and the chat agent's analyze_image tool, so results are consistent between the two entry points.

FR-08

Name	Chat with AI Assistant
Description	Converse with an LLM-driven assistant that can call generation tools (write_script, generate_image, search_stock_images, analyze_image, generate_video) on the user's behalf.
Input	message, conversationId (optional), mode, capability, optional imageUrl
Output	assistant reply text plus a record of any tool calls executed and their results
Basic Work Flow	User sends a message -> AgentService loads conversation history -> the LLM is called with the available tools for the selected capability -> if it requests a tool call, the tool executes (e.g. enqueuing a video job) and its result is fed back to the LLM for a natural-language summary -> both the user message and the assistant's reply are persisted as ChatMessage documents.

FR-09

Name	View History / Dashboard
Description	List the current user's generated videos, images, and analyses with pagination and search.
Input	page, limit, optional search query
Output	a paginated list of items with total counts
Basic Work Flow	Dashboard requests /api/video/list or /api/history/images (or /analyses) -> the controller clamps the requested limit (max 100) and applies the owner filter with a compound index on (owner, createdAt) -> results are returned newest-first.

FR-10

Name	Delete Item
Description	Permanently delete a video, image, or analysis belonging to the current user.
Input	item id
Output	success confirmation
Basic Work Flow	User confirms deletion in the UI -> the corresponding controller (VideoController.deleteVideo, HistoryController.deleteImage/deleteAnalysis) verifies ownership, deletes the database record, and best-effort removes the associated Cloudinary asset -> a rate limiter caps delete requests per user to prevent abuse.

FR-11

Name	Download Item
Description	Download a generated video, image, or PDF analysis report to the user's device.
Input	item id / delivery URL
Output	the binary file streamed to the browser
Basic Work Flow	User clicks Download -> frontend fetches the authenticated delivery URL (Cloudinary signed URL, or a media-token-scoped local URL) -> for analysis reports, a PDF is generated client-side/server-side with the labels, description, and image before download.

FR-12

Name	Manage Queue (Background Jobs)
Description	View, poll, and cancel/remove background jobs (video generation, image generation, editor re-renders).
Input	none (list), or a job id (get/delete)
Output	a list of the user's jobs with status, progress percentage, and stage description
Basic Work Flow	Frontend polls GET /api/jobs every few seconds -> JobController returns the owner's jobs sorted newest-first, projecting out heavy fields like payload.scenes

	-> a job in `processing` shows live progress written by the worker via mapProgressEvent; a completed job's result carries the final videoUrl/imageUrl for the UI to display.

FR-13

Name	Change TTS Voice
Description	Choose which text-to-speech provider/voice narrates a generated or edited video.
Input	ttsProvider ('gtts', 'say', 'elevenlabs-male', 'elevenlabs-female')
Output	narration audio generated with the chosen voice, or a transparent fallback to Google TTS if the chosen ElevenLabs voice is unavailable
Basic Work Flow	User selects a voice in the Studio or Editor -> the choice is passed through the generation/re-render payload -> TTSService.generateSpeech dispatches to the matching provider, caching output by a hash of the text and voice so repeated narration of the same line is not re-synthesised.

FR-14

Name	Change Password
Description	Update the account password from Settings while signed in, without going through the forgot-password/email flow.
Input	currentPassword, newPassword
Output	a success message, the refreshed user profile, and a new access/refresh token pair for the requesting device
Basic Work Flow	User submits the current and new password from Settings -> UserController.changePassword rejects a new password identical to the current one, then verifies the current password against the stored bcrypt hash -> on success the new password is hashed and saved, and the account's tokenVersion is incremented, which immediately invalidates every other outstanding session -> a fresh token pair is issued so the device making the change stays signed in.
Alternate Flow	A Google-only account (no password set) is rejected with a GOOGLE_ACCOUNT error explaining there is no FRAME Studio password

	to change; an incorrect current password is rejected with INVALID_PASSWORD.

FR-15

Name	Sign Out Everywhere
Description	Immediately revoke every active session for the account from a single device, e.g. after suspecting unauthorised access.
Input	none (uses the authenticated session's own identity)
Output	a success confirmation; every previously issued refresh token for the account stops working
Basic Work Flow	User clicks "Sign out everywhere" in Settings -> AuthController.logoutAll calls revokeAllSessions for the current user id -> all stored/valid refresh sessions for that user are revoked, so any other device holding an old token is forced to log in again on its next request.

2.2.3 User Characteristics

The primary user is assumed to be a content creator, student, marketer, or hobbyist with no video editing background, comfortable with a typical web application but not expected to know ffmpeg, prompt engineering conventions, or video codecs. No administrative role beyond the default 'user' is exercised in the current feature set, though the User model reserves an 'admin' role for future use.

2.2.4 Constraints

- Video length is bounded to 1-20 minutes; this bound is enforced server-side (clamped in the job worker) regardless of what a client or an LLM tool call requests.
- Generation quality and availability depend on third-party API uptime and quota; the fallback chains reduce but do not eliminate this dependency.
- ffmpeg must be present on the host with libass and freetype support for subtitle burning and drawtext filters; the Docker image build fails fast if the installed ffmpeg lacks these filters.
- The in-memory rate limiter is per-process, which is an accepted constraint for the current single-instance deployment target and is documented as such in the code.

2.2.5 Assumptions and Dependencies

- A MongoDB instance (Atlas or self-hosted) is reachable and its URI is configured.
- At least one text-generation provider (DeepSeek, Groq, or Gemini) has a valid API key; the system degrades feature-by-feature as keys are removed rather than refusing to boot.
- Cloudinary credentials are configured for durable media storage; without them, generated files remain only in local scratch directories that are not guaranteed to survive a restart.
- The client browser supports EventSource (SSE) for live progress and standard Fetch/XHR with Bearer token headers.

2.2.6 Apportioning of Requirements

Requirements FR-01, FR-02, FR-09, FR-10, and FR-11 are core account/data-management requirements delivered in the current release. FR-03 through FR-08 and FR-12/FR-13 constitute the AI generation and editing feature set, also fully delivered in the current release. Deferred to future work (Chapter 9): multi-language narration/UI, a dedicated mobile client, and an administrative role with cross-user visibility.

2.3 Specific Requirements

2.3.1 Functional Requirements

The functional requirements are as specified in section 2.2.2 (FR-01 through FR-13) and are elaborated further as use cases in Chapter 3.

2.3.2 Non-Functional Requirements

The non-functional requirements below are organised around the quality characteristics defined in ISO/IEC 25010 [2] (functional suitability, usability, reliability, performance efficiency, portability, and maintainability), grouped under the specific headings most relevant to what was actually built rather than reproduced as an exhaustive checklist against every characteristic in the standard.

Usability: The UI groups generation and editing into clearly separated, task-focused pages (Studio, Video Editor, Image Studio, Image Analyzer, Chat) rather than one overloaded screen, and long-running actions surface a visible Queue panel with live progress and stage text (e.g. "Composing segment 3 of 12") rather than a bare spinner.

Reliability: This is the area where the system's design invests the most, because every core feature depends on an external API that can fail independently. Text generation, TTS, and

image generation each have an explicit, ordered fallback chain (documented in Chapter 6) so a single provider outage degrades rather than breaks a feature. Background jobs use lease-based atomic claiming (Models/Job.mjs's workerId/leaseExpiresAt fields) so a server restart mid-render recovers the job exactly once instead of either losing it or double-processing it across two instances. Bounded retry (MAX_ATTEMPTS = 3) prevents a permanently broken job from being retried forever on every boot.

Performance: Per-user daily quotas (dailyQuota middleware) and per-endpoint rate limits (e.g. 30 image generations per 10 minutes, 10 editor re-renders per 10 minutes) bound the load any single account can place on the third-party APIs and on the ffmpeg-bound render workers. Video composition is CPU-intensive, so the three job workers (video, image, render) run as independent single-concurrency slots per process rather than unboundedly in parallel, keeping one process from being overwhelmed by concurrent ffmpeg invocations. History queries use a compound index on (owner, createdAt) so pagination stays fast as a user's library grows, and result pages are capped at 100 items regardless of what a client requests.

Design Constraints: The backend is written in ES modules (.mjs) throughout, chosen for consistency with modern Node.js tooling and native top-level await support used in a few startup paths. All provider-specific configuration lives in one validated config module (config/env.mjs) rather than scattered process.env reads, so a missing key fails predictably rather than silently at the point of use.

Portability: The entire stack is containerised with Docker: a backend image built on node:22-bookworm-slim with ffmpeg/libass/fonts installed at build time (and a build-time check that the installed ffmpeg actually has the required filters), and a frontend image that builds the Vite bundle and serves it through nginx, which also reverse-proxies API and media requests to the backend container so the browser only ever talks to one origin. docker-compose.yml wires the two together with named volumes for generated media so a container rebuild does not discard in-progress work.

Maintainability: Business logic is consistently separated into Services, with Controllers kept thin (parsing the request, calling a service, shaping the response). Several services expose pure, side-effect-free helper functions specifically so they can be unit tested without a database or a live API call (e.g. normalizeEditRenderPayload, sentenceCountForVideoLength, mapProgressEvent) — this is also why the backend's 248 automated tests can run without any external service being reachable.

License Agreement: The project uses standard open-source dependencies (Express, React, Mongoose, fluent-ffmpeg, etc.) under their respective permissive licenses (MIT/Apache-2.0 in the large majority of cases). No proprietary code was incorporated. Third-party AI APIs (DeepSeek, Groq, Gemini, ElevenLabs, Hugging Face, Google Cloud Vision, Pexels) are used under their published API terms of service, each requiring its own API key issued to the operator of the deployment.

CHAPTER 3:
USE CASE ANALYSIS

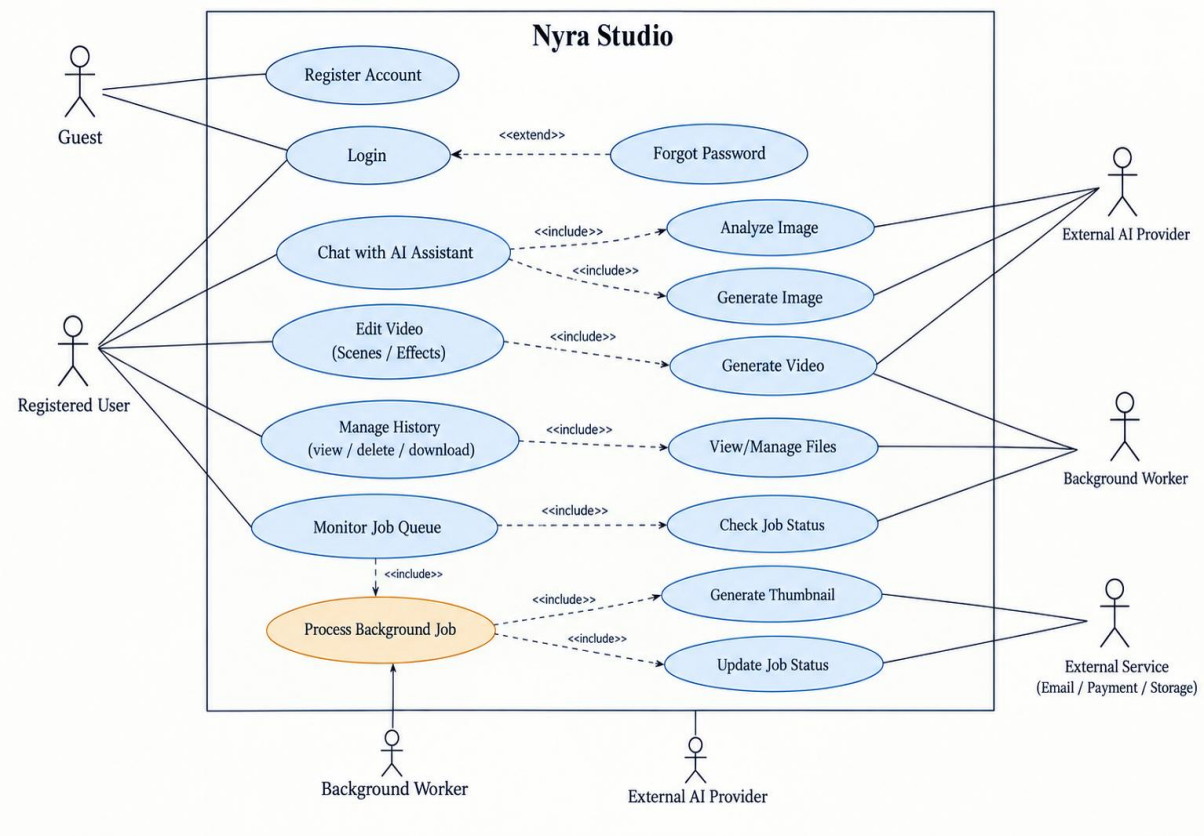


Figure 1: Use Case Diagram - FRAME Studio

Use Case Diagram Description

The aggregated use case diagram for FRAME Studio shows four actors: Guest, Registered User, Background Worker, and two secondary systems reached only through include relationships — External AI Provider and External Service. The Guest actor can only reach Register Account and Login — everything else requires an authenticated session. The Registered User actor is connected to the bulk of the use cases: Login, Chat with AI Assistant, Edit Video, Manage History, and Monitor Job Queue, each of which runs under a verified JWT session and is scoped to that user's own data. Several of these are drawn with an include relationship into the concrete step that actually does the work: Chat with AI Assistant includes Analyze Image and Generate Image (the chat tool-calling loop invokes the same generation pipelines the dedicated pages use); Edit Video includes Generate Video, since a scene re-render reuses the same composition pipeline as a first-time generation; and Monitor Job Queue includes Process Background Job, which is the shared use case owned by the Background Worker actor that actually executes queued video/image/render work asynchronously rather than inside the HTTP request. Process Background Job in turn includes Update Job Status (the worker always writes the job's final status/result back to the database when it finishes) and Generate Thumbnail, which is drawn as part of the same job-lifecycle family because it is the same pattern of asynchronous, on-demand work — the poster frame for a video's history card is cut by ffmpeg the first time it is requested, cached on the Video document, and never regenerated on later loads, rather than being produced eagerly by every render. The External AI Provider is reached by every use case that calls out to a text, image, or vision model (Generate Video, Generate Image, Analyze Image), and the External Service actor represents Resend, the email provider used for OTP delivery during account verification. Two account-security use cases important enough to document in this chapter — Change Password and Sign Out Everywhere (Chapter 2, FR-14/FR-15) — are not drawn as separate lanes in this figure because they are self-contained actions reachable from Settings with no worker or external-provider involvement of their own; they are still fully specified as UC-09 and UC-10 below.

USE CASE 1: Register Account

ID	UC-01
Name	Register Account

Description	A new user creates an account with a username, email, and password, and verifies ownership of the email via a one-time code before the account becomes usable.
Primary Actor	Guest
Secondary Actor	Email delivery service (Resend)
Pre-Condition	The user does not already hold a verified account with the given email.
Post-Condition	A User document exists with emailVerified = true and the user holds a valid access/refresh token pair.
Basic Flow	1. Guest submits username, email, and password on the Sign Up page. 2. System checks for an existing account with that email. 3. System creates the account with a bcrypt-hashed password and a hashed 6-digit OTP, and emails the code. 4. Guest enters the code on the Verify Email screen. 5. System checks the code against the stored hash and expiry, marks the account verified, and issues a JWT pair.
Alternate Flow	A1. If the email belongs to an already-verified account, the system does not reveal this and instead sends a "someone tried to sign up with your email" notice, responding with the same shape as a fresh signup so email enumeration is not possible. A2. If the email belongs to an unverified account (e.g. the user closed the tab mid-signup), the system re-issues a fresh OTP for that same account rather than creating a duplicate. A3. If the code is wrong five times, the code is discarded and the user must request a new one via Resend OTP.

USE CASE 2: Login

ID	UC-02
Name	Login
Description	A registered, verified user authenticates with email/password or a Google

	account and receives a session.
Primary Actor	Registered User
Secondary Actor	Google OAuth (for Google sign-in)
Pre-Condition	The user has a verified account.
Post-Condition	The user holds a fresh JWT access/refresh pair carrying the account's current tokenVersion.
Basic Flow	1. User submits email and password. 2. System looks up the account and compares the password against the stored bcrypt hash. 3. If the account is verified, System updates lastLoginAt and issues a JWT pair. 4. Client stores the tokens and attaches the access token as a Bearer header on subsequent requests.
Alternate Flow	A1. Wrong password or unknown email both return an identical "Invalid credentials" message, so an attacker cannot distinguish a wrong password from a nonexistent account. A2. An unverified account is not logged in; instead a fresh OTP is sent and a 403 EMAIL_NOT_VERIFIED response redirects the client to verification. A3. Google sign-in verifies the ID token's signature, audience, and expiry through google-auth-library, then finds or creates a matching User record instead of a password check.

USE CASE 3: Generate Video

ID	UC-03
Name	Generate Video
Description	A user generates a narrated video from a topic, a length, an image source, and a voice choice.
Primary Actor	Registered User
Secondary Actor	System/Background Worker; third-party AI providers (LLM, TTS, image)

Pre-Condition	User is authenticated and has not exceeded the daily video generation quota.
Post-Condition	A completed Video document exists with a Cloudinary-hosted URL, scene data, and subtitle text, or the job is marked failed with a stored error.
Basic Flow	1. User enters a topic, selects a length (1-20 minutes), an image source, and a TTS voice in the Studio. 2. System checks the daily quota and rate limit, then enqueues a 'video' job. 3. Background Worker claims the job, generates the script, generates/sources one image per scene, synthesises narration per scene, and composes the final video with ffmpeg (transitions, subtitles, background music). 4. Worker uploads the result to Cloudinary, creates the Video document, and marks the job completed. 5. Client, polling the Queue panel, sees the completed job and displays the finished video.
Alternate Flow	A1. If the primary text provider fails, the system retries with the next provider in the fallback chain before failing the script step. A2. If the chosen ElevenLabs voice is unavailable (quota or invalid key), narration silently falls back to Google TTS so the video still completes. A3. If an image provider fails for a scene, a deterministic placeholder frame is used instead of dropping the scene, so the video's length is not silently shortened. A4. If the job crashes mid-render, it is retried up to two more times before being marked permanently failed.

USE CASE 4: Edit Video (Scenes / Effects)

ID	UC-04
Name	Edit Video
Description	A user opens a previously generated video in the Video Editor and either edits individual scenes (narration text, image, duration, transitions) or applies visual effects, producing a new rendered video.
Primary Actor	Registered User

Secondary Actor	System/Background Worker
Pre-Condition	The user owns the video being edited.
Post-Condition	A new Video document is created representing the re-rendered result; the original video is left untouched.
Basic Flow	1. User opens the editor for a video, which loads its persisted scene list. 2. User edits a scene's subtitle text, replaces its image, or reorders/adjusts transition timing, or selects one or more visual effects from the Effects tab. 3. User clicks Save Edit (or Apply Effects); the request is queued as a 'render' job. 4. Worker re-renders: for a scene edit, only scenes whose text changed are re-narrated, all others reuse existing narration; for effects, the ffmpeg filter chain for the selected effects is applied to the whole video. 5. Result is uploaded and a new Video document is created; the editor claims the finished job by matching its sourceVideoId.
Alternate Flow	A1. If the effects payload requests more than 20 effects or references an invalid videoId, the request is rejected before it reaches the queue. A2. If the video's source is a remote URL, the system validates it is https/http, not a redirect, under a size ceiling, and from an explicitly allowed host, before downloading it, to prevent SSRF. A3. A synchronous (non-queued) fallback endpoint exists for both scene edits and effects in case the background queue path is unavailable.

USE CASE 5: Generate Image

ID	UC-05
Name	Generate Image
Description	A user generates a single AI image from a text prompt.
Primary Actor	Registered User
Secondary Actor	Third-party image generation providers
Pre-Condition	User is authenticated and within the daily image quota.

Post-Condition	An Image document and Asset record exist in the user's library, or the image is returned without being persisted if storage failed.
Basic Flow	1. User enters a prompt in the Image Studio. 2. System checks quota/rate limit and calls the image generation pipeline. 3. The pipeline tries providers in order (Flux, then Gemini, then others) until one succeeds. 4. The generated file is uploaded to Cloudinary; an Asset and an Image document are created. 5. The delivery URL is returned to the user and shown in the gallery.
Alternate Flow	A1. If every provider fails, the system reports which providers were tried and a specific, actionable hint (e.g. "Gemini's free tier allows 0 image requests — enable billing or use another provider"). A2. If generation succeeds but the Cloudinary upload fails, the image is still returned to the user with persisted:false and a warning that it will not be kept.

USE CASE 6: Analyze Image

ID	UC-06
Name	Analyze Image
Description	A user uploads a photo and receives labels, a description, and suggested follow-up prompts.
Primary Actor	Registered User
Secondary Actor	Vision-capable LLM provider; optionally Google Cloud Vision
Pre-Condition	User is authenticated.
Post-Condition	An Analysis document is persisted in the user's history.
Basic Flow	1. User uploads an image file on the Image Analyzer page. 2. System sends the image to a vision-capable LLM with a structured prompt requesting JSON output (labels, description, three suggested prompts). 3. If Google Cloud Vision is configured, its labels are merged in to enrich (not

	replace) the LLM's labels. 4. The image is uploaded to Cloudinary and the Analysis document is saved. 5. Result is displayed, with an option to export a PDF report or send a suggested prompt straight to the Image or Video studio.
Alternate Flow	A1. If the model's JSON response is unusable, a fallback description and generic suggested prompts are built from whatever labels were extracted, rather than returning nothing. A2. The same analysis function is used whether the request comes from the page or from the chat agent's analyze_image tool, so results are consistent regardless of entry point.

USE CASE 7: Chat with AI Assistant

ID	UC-07
Name	Chat with AI Assistant
Description	A user converses with an LLM assistant that can call generation tools on their behalf.
Primary Actor	Registered User
Secondary Actor	LLM provider; System/Background Worker (for queued tool calls like generate_video)
Pre-Condition	User is authenticated and within the chat rate limit.
Post-Condition	A ChatMessage pair (user + assistant) is persisted under a Conversation; any invoked tool's side effects (a queued job, a saved image, a saved analysis) exist independently of the chat transcript.
Basic Flow	1. User sends a message in the Chat page, optionally in a restricted "capability" mode (e.g. only image generation). 2. System loads the conversation history and calls the LLM with the available tool definitions. 3. If the LLM decides to call a tool (e.g. generate_video), the system executes it and feeds the result back to the LLM to produce a natural-language summary.

	4. The reply and any tool call records are saved and returned to the user.
Alternate Flow	A1. In "tool" mode, the assistant is instructed to clarify missing details (length, visual style) and restate the plan before calling generate_video or generate_image, rather than acting immediately. A2. If a tool throws an error, the error is captured as the tool's result and summarised to the user rather than failing the whole chat turn.

USE CASE 8: Manage History (View / Delete / Download)

ID	UC-08
Name	Manage History
Description	A user views, searches, paginates, deletes, and downloads previously generated videos, images, and analyses.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User is authenticated.
Post-Condition	The requested item is deleted (for delete) or downloaded to the user's device (for download); no state changes for view/search.
Basic Flow	1. User opens the Dashboard or Studio library view. 2. System returns a paginated, newest-first list of the user's items using an indexed query on (owner, createdAt). 3. User may search/filter, delete an item (with ownership re-verified server-side before deletion), or download an item. 4. Deletion removes the database record and best-effort removes the associated Cloudinary asset.
Alternate Flow	A1. A requested page/limit outside sane bounds is clamped server-side (limit capped at 100) rather than trusted as given. A2. Deleting an item the user does not own returns 404, not a silent no-op or a 403 that would confirm the item's existence.

USE CASE 9: Change Password

ID	UC-09
Name	Change Password
Description	A signed-in user changes their account password from Settings and every other active session is signed out as a result.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User is authenticated and has a password set on the account (not a Google-only account).
Post-Condition	The account's password hash is updated, its tokenVersion is incremented (invalidating every other session's tokens), and the requesting device is issued a fresh token pair.
Basic Flow	1. User opens Settings and enters their current password and a new password. 2. System verifies the current password against the stored bcrypt hash. 3. System hashes and saves the new password and increments the account's tokenVersion. 4. System issues a fresh access/refresh token pair to the requesting device and confirms the change.
Alternate Flow	A1. If the new password is identical to the current one, the request is rejected with SAME_PASSWORD before any comparison against the stored hash. A2. If the account has no password (a Google-only account), the request is rejected with GOOGLE_ACCOUNT rather than attempting a comparison. A3. If the current password does not match, the request is rejected with INVALID_PASSWORD and no change is made.

USE CASE 10: Sign Out Everywhere

ID	UC-10
Name	Sign Out Everywhere

Description	A signed-in user revokes every active session on every device from Settings on the current device.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User is authenticated.
Post-Condition	Every previously issued refresh session for the account is revoked; any other device must sign in again on its next authenticated request.
Basic Flow	1. User clicks "Sign out everywhere" in Settings. 2. System revokes every stored refresh session associated with the user's account. 3. System confirms the action to the requesting device.
Alternate Flow	A1. The action does not require re-entering a password, since it only needs the caller's own already-valid session to authorise it.

USE CASE 11: Monitor Job Queue

ID	UC-11
Name	Monitor Job Queue
Description	A user watches the live status of their own queued and in-progress background jobs (video generation, image generation, editor re-renders) and can cancel or remove a job entry.
Primary Actor	Registered User
Secondary Actor	System/Background Worker
Pre-Condition	User is authenticated.
Post-Condition	The Queue panel reflects each job's current status, progress percentage, and stage text; a removed job no longer appears in the list.
Basic Flow	1. User opens the Queue panel. 2. Frontend polls GET /api/jobs every few seconds. 3. System returns the owner's jobs, newest first, including status

	(queued/processing/completed/failed), a progress percentage, and a human-readable stage (e.g. "Composing segment 3 of 12"). 4. Background Worker updates a job's progress and stage as it works through the pipeline, which the next poll picks up. 5. User may delete a completed or failed job entry to clear it from the list.
Alternate Flow	A1. A completed job's entry carries the final videoUrl/imageUrl so the UI can link directly to the finished result without a separate lookup. A2. Deleting a job the user does not own returns 404, the same ownership check used by Manage History.

CHAPTER 4:
DESIGN

The diagrams referenced in this chapter are inserted as figures alongside the explanatory text below. Each subsection describes what the corresponding diagram shows and why it is structured the way it is, grounded in the actual modules and classes present in the codebase.

Architecture Diagram

The architecture diagram shows FRAME Studio as a three-tier system: a React/Material-UI single-page frontend, an Express.js REST API backend, and MongoDB for persistence, with a distinct fourth layer of external services the backend depends on. The frontend never talks to third-party AI providers, Cloudinary, or MongoDB directly — every request goes through the Express API, which is the only component holding provider API keys and database credentials. Inside the backend, the diagram separates Routes (thin HTTP-to-controller wiring plus middleware composition), Controllers (request parsing and response shaping), Services (the actual business logic — script generation, TTS, image generation, video composition, job orchestration), and Models (Mongoose schemas). A distinct box represents the background Job Worker, which runs inside the same Node process as the API but on its own polling loop (three independent worker slots for video, image, and render jobs), and is drawn separately because it is what actually executes long-running work the HTTP handlers only enqueue. The diagram is structured this way — rather than, for example, drawing the worker as part of the API layer — because the lease-based claiming logic in JobService.mjs is written to be safe even if the worker eventually runs as a separate process or a second replica, which the current single-process design does not yet require but is deliberately compatible with.

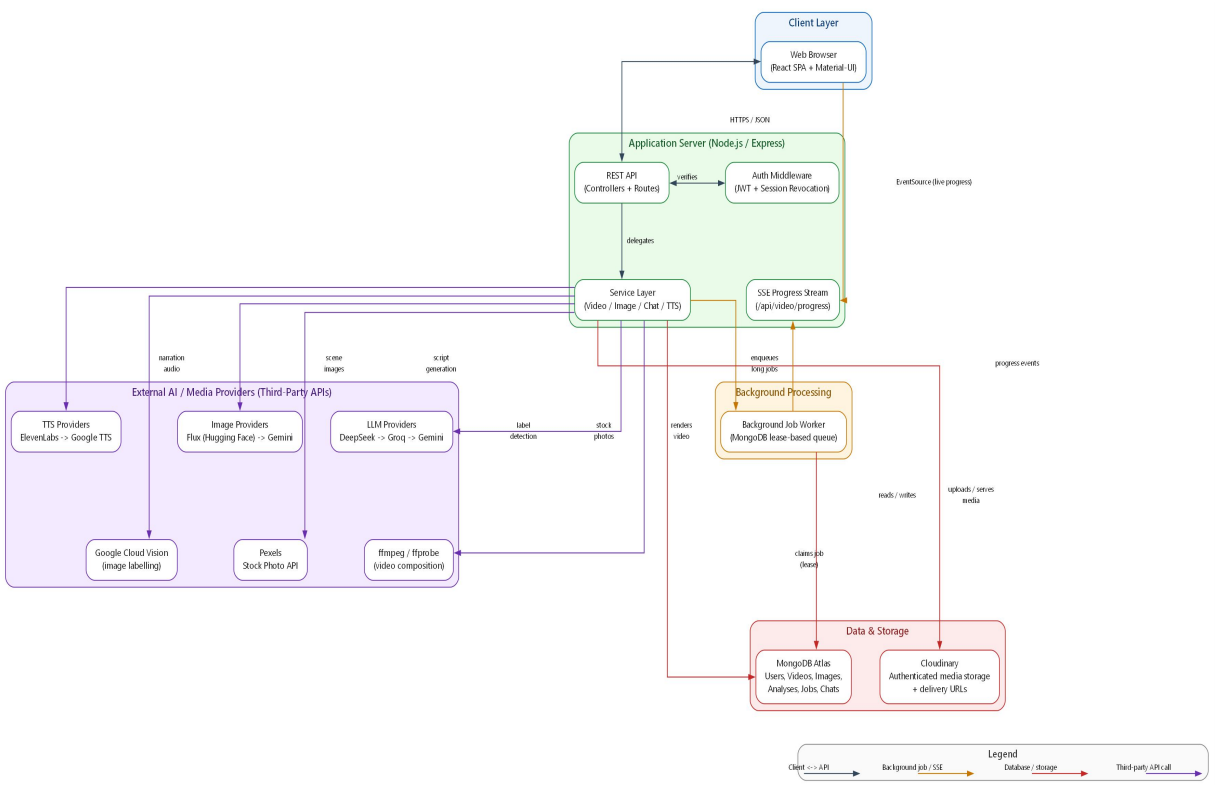


Figure 2: System Architecture Diagram

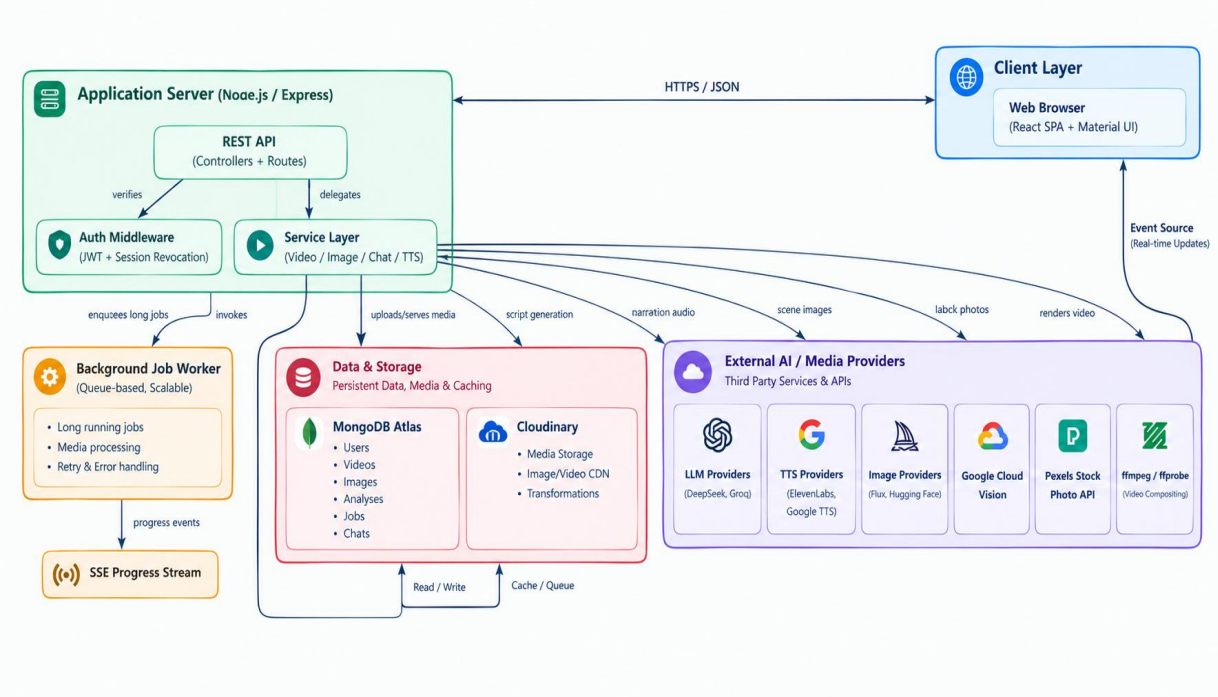


Figure 2b: System Architecture Diagram (reference rendering)

Entity Relationship Diagram (ERD)

The ERD is built directly from the eight Mongoose models in the backend: User, Video, Image, Analysis, Asset, Job, Conversation, and ChatMessage. User is the root entity that everything else references through an `owner` field, and every content-bearing collection carries at least an indexed `owner` reference so per-user queries never require a full collection scan. Five of these collections are listed back to their owner sorted by recency and carry a compound index matching that exact query shape: Video, Image, Analysis, and Job use `(owner, createdAt)` for the history/dashboard "newest first" pagination described in Chapter 5, and Conversation uses `(owner, updatedAt)`, since AgentService.listConversations sorts by when a chat was last active rather than when it was first created. Asset is the one collection never listed this way — it is only ever looked up by its own `_id` (with an owner check for authorization) from the Video or Image document that references it, so a plain single-field `owner` index is all it needs. Video and Image both optionally reference an Asset (the durable Cloudinary-backed record of bytes/format/dimensions) and a Job (the background job that produced them), reflecting that a piece of generated content is the persisted result of a unit of background work, not something created inline in the request that asked for it. Video additionally embeds its own `scenes` array directly on the document rather than as a separate collection, because scenes are always read and written as a unit with their parent video (the editor loads and saves the whole list at once) and never queried independently. Conversation and ChatMessage form a one-to-many relationship (a conversation has many messages), and each ChatMessage optionally carries a `toolCalls` array recording what tools the agent invoked and their results, which is what lets a past conversation be replayed with its generation history intact.

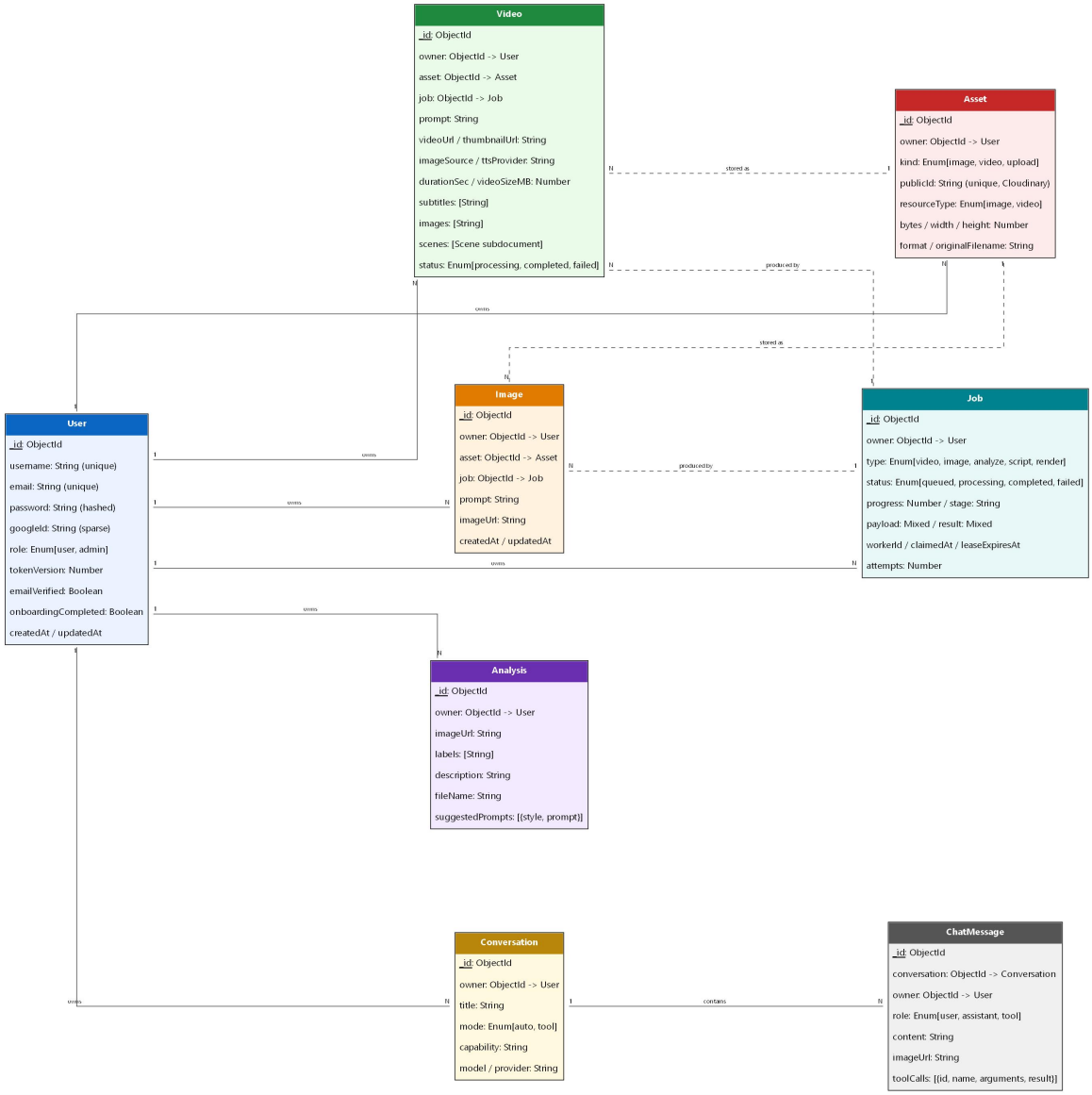


Figure 3: Entity Relationship Diagram (ERD)

DFD Level 0 (Context Diagram)

The Level 0 diagram treats the whole FRAME Studio backend as a single process and shows its data flows to and from four external entities: the User (via the browser), the Database (MongoDB), Cloud Storage (Cloudinary), and third-party AI Providers (collectively, since at this level the specific provider is not yet relevant). The User sends prompts and receives generated content and status updates; the system reads and writes user accounts, content records, and job state to the Database; it uploads finished media to and receives signed URLs from Cloud Storage; and it sends generation requests to, and receives generated text/audio/images from, the AI Providers. This diagram deliberately does not distinguish between the seven or eight actual third-party APIs used, because at the context level what

matters is that the system is a broker between a user and a family of external generation services, not which specific vendor is called for which step.

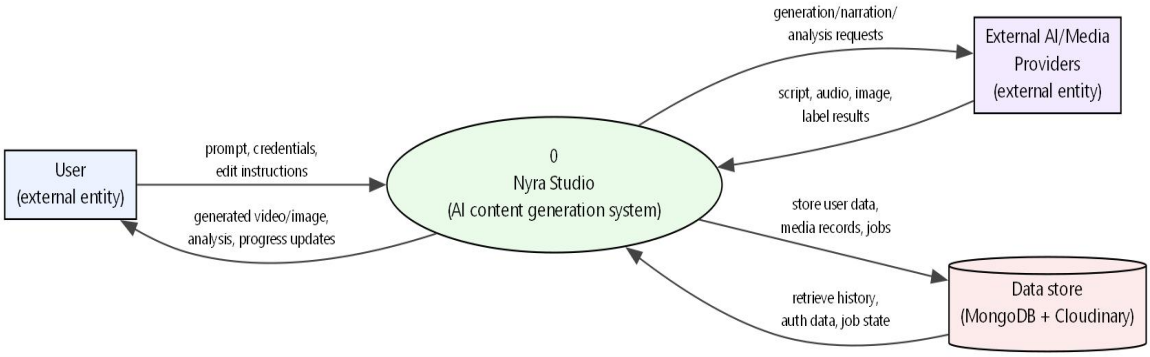


Figure 4: Data Flow Diagram - Level 0

DFD Level 1

The Level 1 diagram decomposes the single process from Level 0 into its major functional processes: Authentication, Video Generation, Video Editing, Image Generation, Image Analysis, Chat/Agent, and Job Queue Management, each shown with its own data flows into and out of the relevant data stores (Users, Videos, Images, Analyses, Jobs, Conversations). The Job Queue Management process is drawn as a hub that several other processes flow through: Video Generation, Video Editing, and (for the editor's per-scene AI image button) Image Generation all write a Job record and hand off to it rather than performing their long-running work inline, which mirrors the actual code path where controllers call `enqueue()` and return immediately with a job id. The diagram also shows Chat/Agent with a flow into Job Queue Management, reflecting that the chat tool `generate_video` enqueues a job exactly the same way the Studio's UI does, rather than running its own separate generation logic.

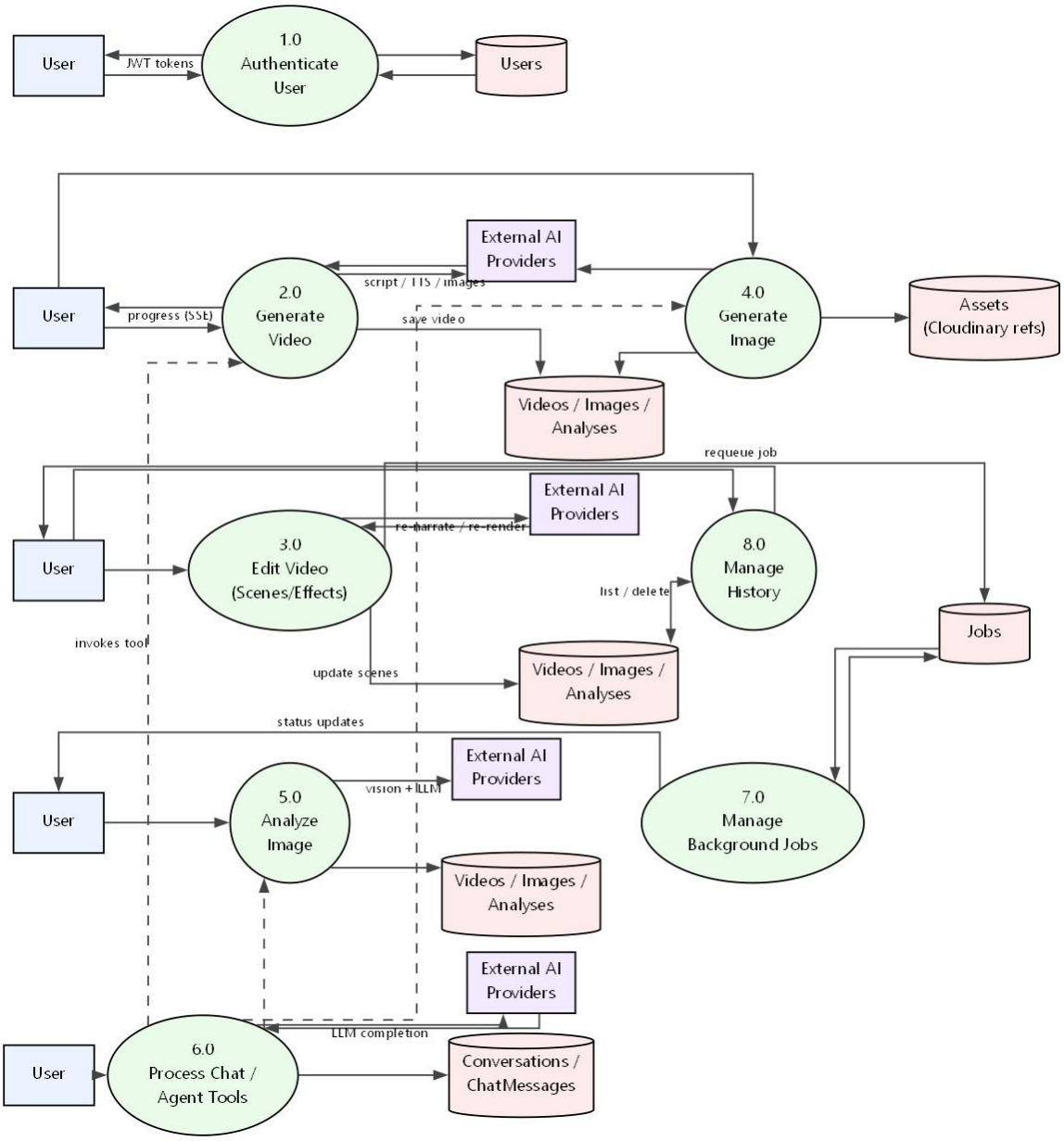


Figure 5: Data Flow Diagram - Level 1

Class Diagram

The class diagram combines the persistent data model with the handful of Service-layer classes that create or mutate it, since drawing the two separately would hide exactly the relationships that matter. User sits at the centre as the ownership hub: every content-bearing class in the diagram — Video, Image, Analysis, Job, Asset, and Conversation — carries a 1-to-N "owns" association back to User, mirroring the `owner` field every one of those Mongoose schemas actually has. Three of these classes embed a smaller class directly rather than referencing a separate top-level collection, shown as UML composition (the filled-diamond end): Video contains Scene (a video's per-scene image/narration/subtitle/duration

data), Analysis contains SuggestedPrompt (the 2-3 follow-up prompts returned with an image analysis), and ChatMessage contains ToolCall (the record of a tool the chat agent invoked and its result). Video and Image separately hold a plain reference to Asset ("stored as"), since an Asset is a shared, independent record of the underlying Cloudinary-hosted bytes rather than something either class owns exclusively. Around this data model, four Service classes are shown with dashed dependency arrows rather than inheritance, since none of them extend one another: VideoGeneratorService creates a Video and uses TTSService for narration; VideoEditService updates an existing Video in place for scene re-renders; and AgentService, the chat orchestration loop, both invokes VideoGeneratorService as a tool call and creates the ChatMessage records for a conversation.

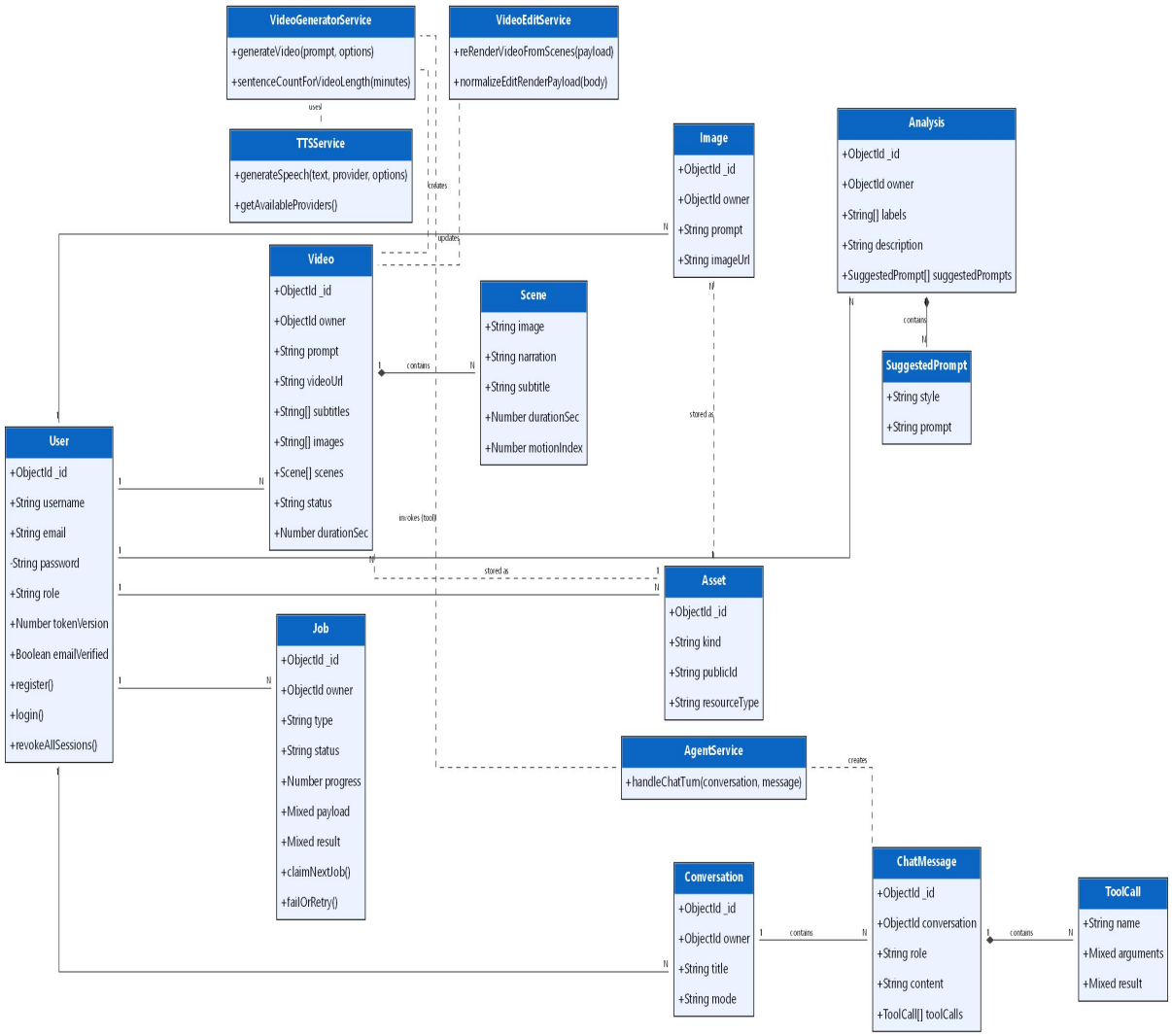


Figure 6: Class Diagram

Activity Diagram

The activity diagram traces the Generate Video use case as a swimlane flow between the User, the API, and the Background Worker. The User's lane is short — submit prompt, then later view the result — while the Worker's lane carries the bulk of the activity: generate script, then for each scene in a loop, generate/source an image and synthesise narration in parallel branches, then a join point before video composition, followed by upload and database persistence. The loop-with-parallel-branches structure mirrors the real implementation, where the per-scene image generation and TTS calls are not strictly sequential from a modelling standpoint even though the current code issues them scene-by-scene; the diagram intentionally shows where the pipeline has parallelisable structure, since this is exactly the kind of optimisation identified for future work in Chapter 9. A decision diamond after each provider call represents the fallback branching (did this provider succeed, or fall through to the next configured provider), which appears at three separate points in the flow — script generation, image generation, and TTS — because all three subsystems implement the same fallback pattern independently.

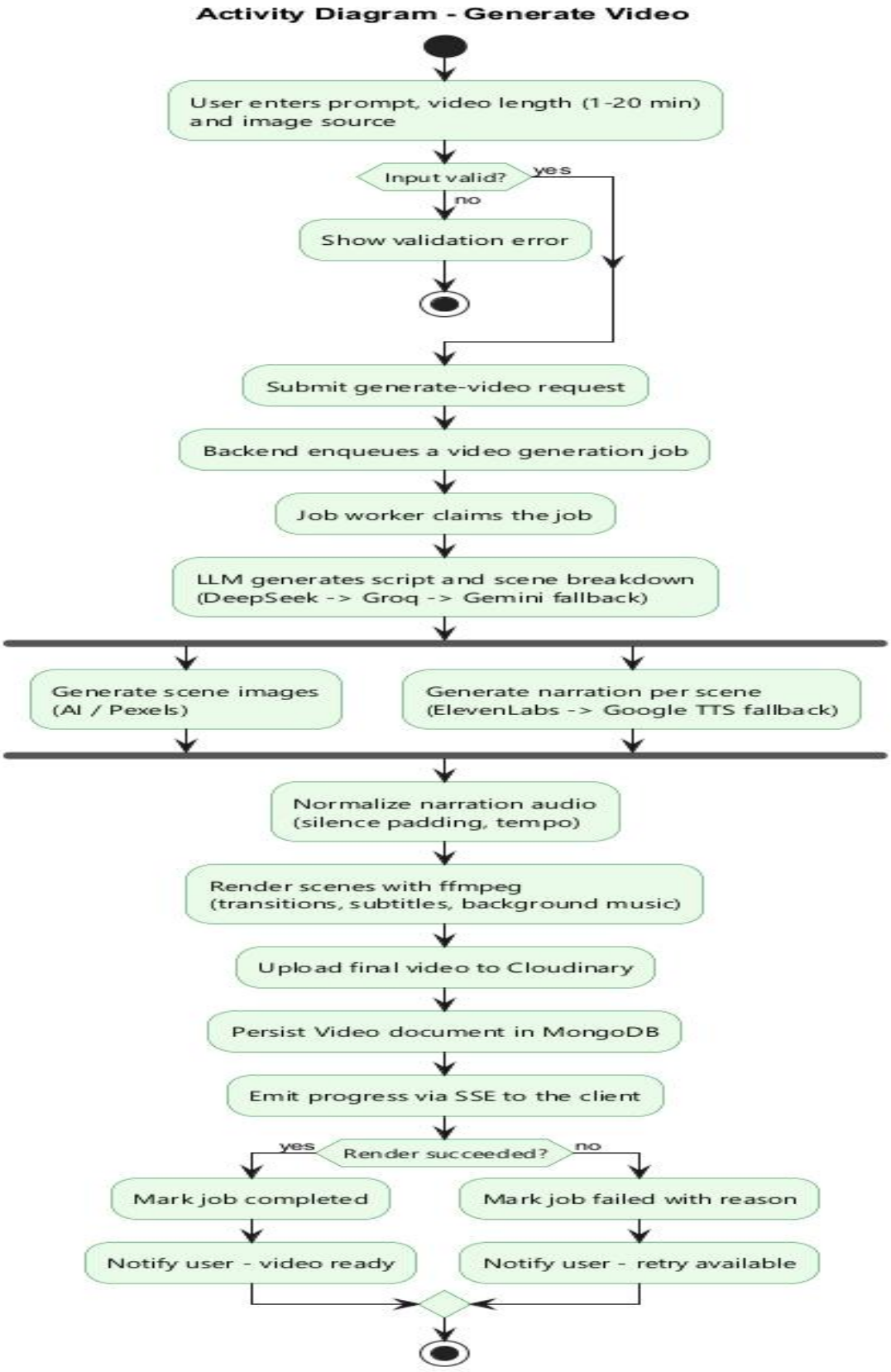


Figure 7: Activity Diagram - Generate Video

Sequence Diagram

The sequence diagram details the Edit Video (Scenes) use case, chosen over a plain generation flow because it best demonstrates the system's surgical re-render design. It shows the Editor UI calling POST /api/video/editor/queue-re-render, the controller calling `enqueue()` to create a Job document and returning a 202 with a jobId immediately, and then a second, independent timeline for the Worker: it claims the job, and for each scene in the payload checks whether that scene's index is in `changedSubtitleIndexes` — if so it calls TTSService to synthesise fresh narration; if not, it reuses the scene's already-persisted narration path from the Video document without touching TTSService or the cache at all. Both branches converge at VideoService.renderFromScenes, which composes the final file from whatever mix of old and new narration clips the loop produced. The diagram is drawn with two lifelines for "the request" and "the worker" specifically to make visible that the HTTP response returns long before the render finishes — a detail that is easy to miss if the diagram is drawn as one linear sequence.

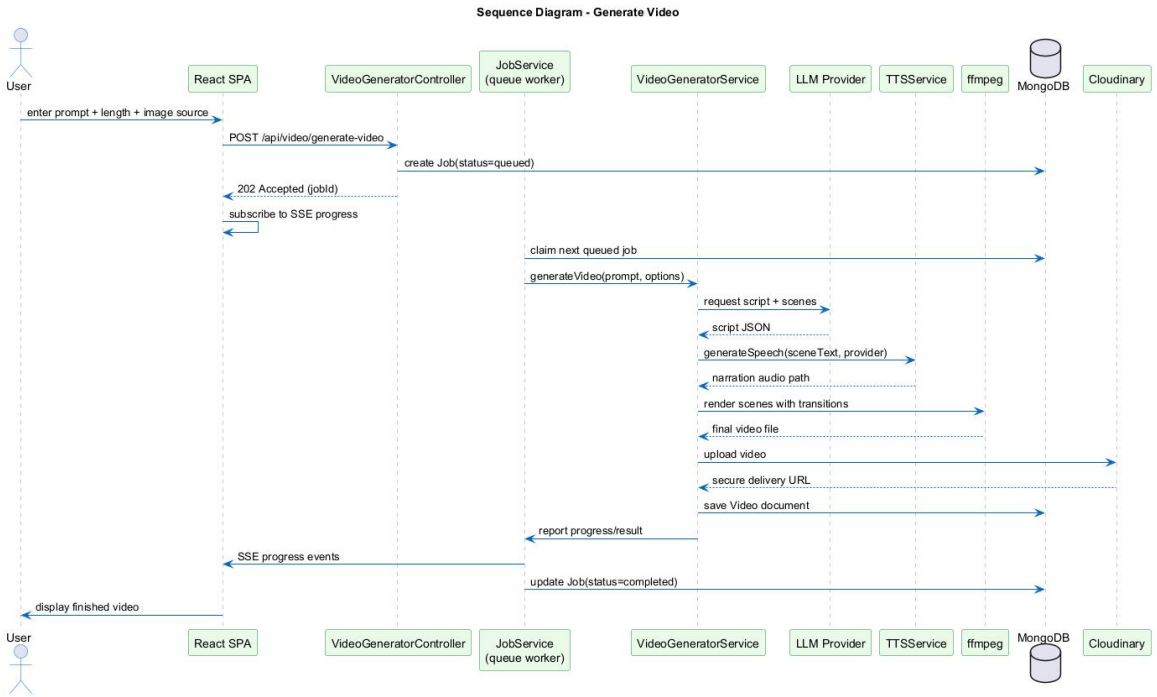


Figure 8: Sequence Diagram - Generate Video

Collaboration Diagram

The collaboration diagram represents the same Generate Video use case as the sequence diagram but organised around the objects and their message links rather than time, to make the object structure explicit. The request chain runs across the top exactly as it does in the

sequence diagram — User submits a prompt to the React SPA, which posts to VideoGeneratorController, which enqueues the work with JobService — but the collaboration view's real value is below that: JobService calls VideoGeneratorService.generateVideo() once (message 4), and every subsequent step (5 through 9) is drawn as a direct link from VideoGeneratorService outward to the five collaborators it calls in turn: the LLM Provider for the script, TTSService for narration, ffmpeg for scene rendering, Cloudinary for the finished upload, and MongoDB to persist the Video document. Laying these five links out radially around VideoGeneratorService, rather than as one long vertical timeline the way the sequence diagram draws them, makes the object structure explicit in a way the sequence diagram cannot: VideoGeneratorService is the single coordinating object for the entire generation pipeline, and none of its five collaborators are ever reachable directly from the controller, the job service, or each other.

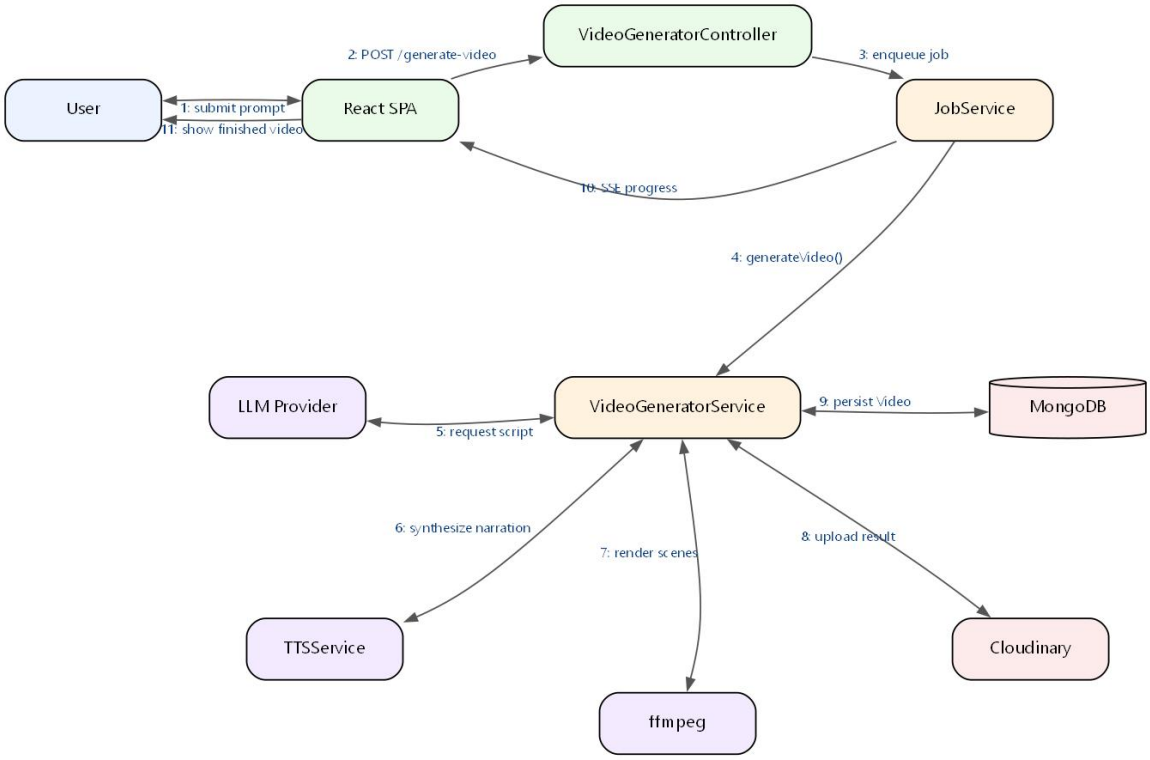


Figure 9: Collaboration Diagram - Generate Video

State Transition Diagram

The state transition diagram models the lifecycle of a Job document, which is the aspect of the system with the clearest, most literal state machine: `queued` -> `processing` -> `completed` or `failed`. The diagram shows the terminal `failed` path explicitly: `processing` moves to `failed` once a job's attempts are exhausted, whether that exhaustion is discovered

from an error thrown during the render or from the startup recovery sweep finding an abandoned job (a worker's lease expired without the job finishing, e.g. a server restart mid-render) whose attempts are already used up. Not shown as a separate branch on this diagram, but implemented identically in JobService.mjs, is the companion case where attempts are not yet exhausted: an error or an expired lease in that case returns the job directly to `queued` for another attempt rather than reaching `failed`, which is why a job can be observed cycling through `queued` and `processing` more than once before it finally resolves to `completed` or `failed`. This diagram is included because the whole reliability story in Chapter 2's non-functional requirements is really a claim about this state machine behaving correctly under concurrent workers and crashes, which is exactly what the lease/heartbeat fields on the Job model (workerId, claimedAt, leaseExpiresAt) exist to guarantee.

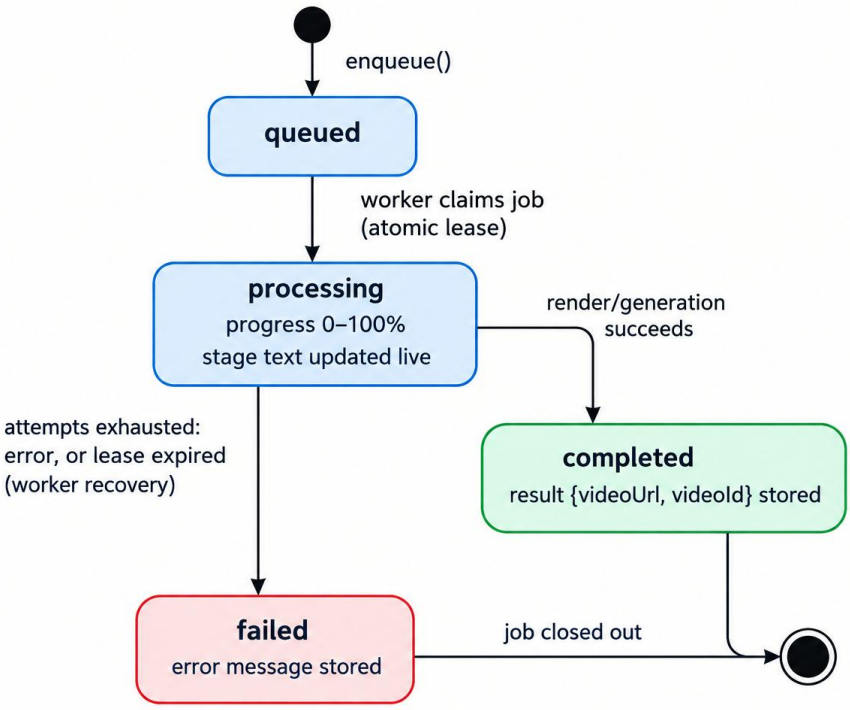


Figure 10: State Transition Diagram - Background Job Lifecycle

Component Diagram

The component diagram groups the backend into four layers — Frontend, Controllers, Services, and Infrastructure — connected through explicit provided and required interfaces (UML's lollipop-and-socket notation) rather than plain arrows, to make clear exactly which component owns which contract. The Frontend SPA depends on a single ControllerAPI socket over HTTP/REST; every Controller then exposes its own provided interface (JobAPI,

ImageAPI, ChatAPI, ImageGenAPI, VideoEditAPI, VideoGenAPI, AuthAPI) and requires the matching interface from its Service. Two controllers deliberately point at the same service surface: JobController and ImageAnalyzerController both require a JobAPI, since analysis results are also tracked through the job queue, and ChatController's ChatAPI is served by AgentService/tools, which is itself the component that requires VideoGenAPI and ImageGenAPI to invoke video and image generation as chat tools — this is the same "agent invokes the generation services on the user's behalf" relationship shown as an association in the use case diagram, now shown as a formal interface dependency. On the Infrastructure side, the diagram distinguishes an assembly connector (a component wired directly to the interface it needs, drawn solid) from a dependency (a looser runtime reliance, drawn dashed): JobService, VideoGeneratorService, VideoEditService/VideoEffectsService, and ImageAnalysisService all depend on Cloudinary SDK for MediaStorageAPI, VideoGeneratorService and VideoEditService/VideoEffectsService additionally depend on fluent-ffmpeg for MediaProcessingAPI, and TTSService depends on Cloudinary for storing synthesised narration; every service that persists a document depends on MongoDB through a shared PersistenceAPI. AgentService/tools and llm/provider.mjs both expose and depend on the same LLMProviderAPI, reflecting that the chat agent's own model calls and the shared fallback-chain provider module are two sides of the same interface rather than separate integrations.

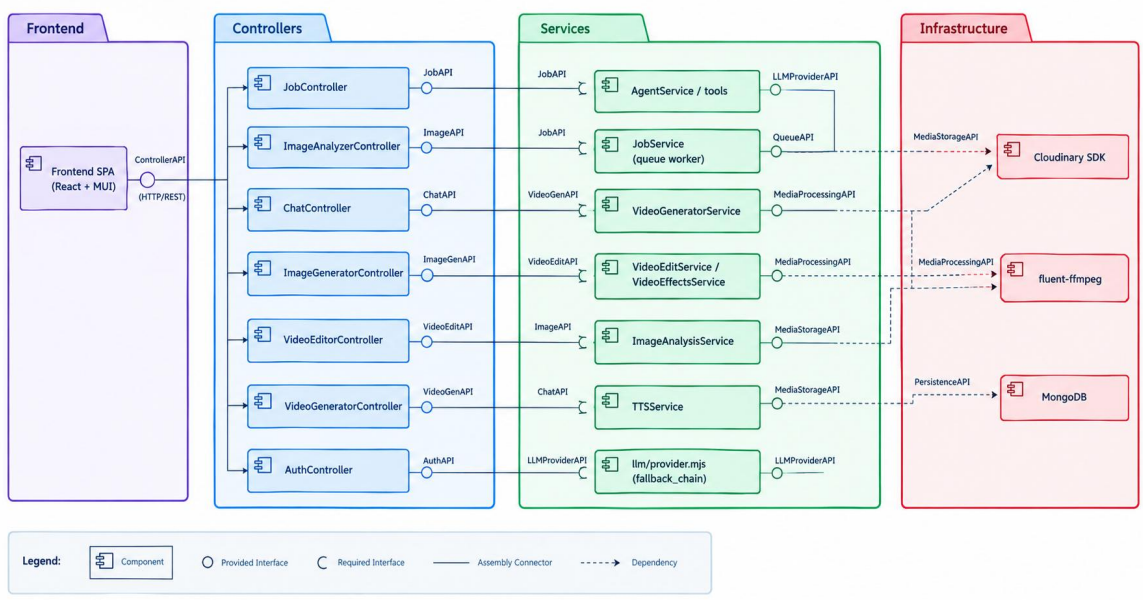


Figure 11: Component Diagram

Deployment Diagram

The deployment diagram shows two containers — backend and frontend — orchestrated by docker-compose, plus the externally-hosted MongoDB (Atlas, or an optional local Mongo container under the `local-db` profile) and the various third-party API endpoints reached over the public internet. The frontend container runs nginx, which serves the built static bundle and reverse-proxies API and media paths to the backend container over the internal compose network, so the browser only ever establishes a connection to one origin. The backend container is shown with named Docker volumes attached for each generated-media directory (downloads, videos, uploads, segments, scene audio, TTS cache), which is deliberate: these volumes are what let a container be rebuilt or redeployed without discarding media that has not yet been uploaded to Cloudinary. The backend's healthcheck (`/api/health`) gates the frontend container's startup order in the compose file, so the frontend is never marked ready while the API it proxies to is still booting. This dockerised topology is the deployment target for a self-managed VM or a future migration off managed platforms, and the Dockerfiles and docker-compose.yml described here and in Appendix A are kept fully working and tested for exactly that purpose, independent of where the project happens to be hosted today.

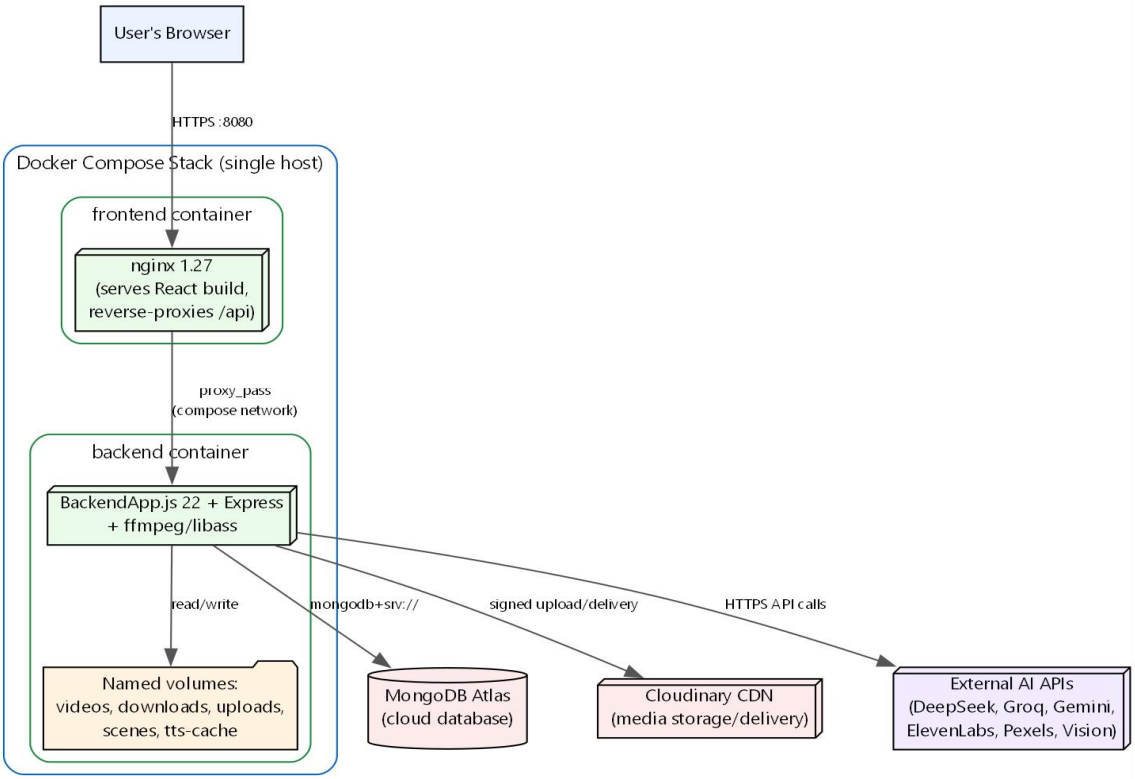


Figure 12: Deployment Diagram

Live deployment. For the current live deployment, the two containers described above are not run together on one host; each half of the stack is instead hosted on its own managed platform, since this avoids paying for and administering a VM for a project at this scale while development is ongoing. The frontend is deployed on Vercel, which builds the React app directly from the repository on every push and serves it from Vercel's own CDN/edge network rather than the nginx container: https://fyp-project-sand.vercel.app/. The backend is deployed on Railway, which builds and runs the same Express/ffmpeg backend image from the repository's Dockerfile, reachable at https://fypproject-production-7f42.up.railway.app/, with its health check available at /api/health exactly as it is in the docker-compose deployment. The frontend's API base URL points at the Railway backend instead of a same-origin nginx proxy, so this deployment relies on the backend's CORS configuration allowing the Vercel origin, rather than the single-origin reverse-proxy arrangement the docker-compose setup uses; everything else — MongoDB Atlas, Cloudinary, and the third-party AI providers — is reached identically from either deployment, since none of that depends on which platform is running the containers.

CHAPTER 5:
TESTING

5.1 Test Case Specifications

The nine test cases below are worked examples of the black-box test design techniques covered in Section 5.2 (equivalence partitioning, boundary value analysis, decision tables, and state transition testing), documented in the university's own test-case format (ID, priority, steps, expected result, status) so the design technique behind each one is traceable on paper. They are a representative sample chosen to cover the highest-risk flows (login, video generation, effects, deletion, and the account-security actions added in Chapter 2), not an enumeration of every test performed. The actual scale of automated testing is the 248-test suite described in Section 5.3.2 and reproduced in full in Appendix B; TC-01 through TC-09 below correspond to a subset of that suite's scenarios, re-expressed in the formal test-case format for this section specifically, rather than being a separate, smaller set of tests.

TC-01

Priority	High
Description	Successful login with correct email and password
Reference	UC-02 Login
Users	Registered User
Pre-requisites	An account exists with emailVerified = true
Steps	1) POST /api/auth/login with the account's email and password. 2) Inspect the response.
Input	{ email: "user@example.com", password: "CorrectPass123" }
Expected result	HTTP 200; response contains a valid access token, refresh token, and user object with emailVerified: true
Status	Pass

TC-02

Priority	High
Description	Failed login with wrong password
Reference	UC-02 Login

Users	Registered User
Pre-requisites	An account exists with emailVerified = true
Steps	1) POST /api/auth/login with the correct email and an incorrect password. 2) Inspect the response.
Input	{ email: "user@example.com", password: "WrongPassword" }
Expected result	HTTP 400; response body { msg: "Invalid credentials" }; no token issued; message identical in wording to an unknown-email attempt (no account enumeration)
Status	Pass

TC-03

Priority	High
Description	Generate video with a valid prompt
Reference	UC-03 Generate Video
Users	Registered User
Pre-requisites	User authenticated; under daily video quota
Steps	1) POST the generation request with a non-empty prompt, a videoLength between 1 and 20, and an imageSource. 2) Poll /api/jobs/:id until status is completed. 3) Inspect the result.
Input	{ prompt: "The history of the Great Wall of China", videoLength: 3, imageSource: "gemini", ttsProvider: "gtts" }
Expected result	Job transitions queued -> processing -> completed; result.videoUrl is a reachable Cloudinary URL; a Video document exists with scenes.length matching the number of narration lines generated
Status	Pass

TC-04

Priority	High

Description	Generate video with an empty prompt (validation)
Reference	UC-03 Generate Video
Users	Registered User
Pre-requisites	User authenticated
Steps	1) POST the generation request with an empty or whitespace-only prompt.
Input	{ prompt: "", videoLength: 3, imageSource: "gemini" }
Expected result	HTTP 400 before any job is enqueued; no Job document is created; error message indicates the prompt is required
Status	Pass

TC-05

Priority	Medium
Description	Apply a visual effect to an existing video
Reference	UC-04 Edit Video
Users	Registered User
Pre-requisites	User owns a completed Video; effect name is one of the values returned by GET /api/video/editor/effects
Steps	1) POST /api/video/editor/queue-apply-effects with videoId and a valid effects array. 2) Poll the job. 3) Inspect the result.
Input	{ videoId: "<valid 24-hex id>", effects: ["cinematic", "colorGrading"] }
Expected result	HTTP 202 with a jobId; job completes with result.kind = "editor-apply-effects" and a new videoUrl distinct from the source video; source video is unmodified
Status	Pass

TC-06

Priority	High

Description	Delete a video the user owns
Reference	UC-08 Manage History
Users	Registered User
Pre-requisites	User owns at least one Video document
Steps	1) DELETE /api/video/:id for an owned video id. 2) GET /api/video/list and confirm it is absent. 3) Attempt to DELETE the same id again.
Input	DELETE /api/video/<owned-id>
Expected result	First delete returns HTTP 200 with success: true and the video no longer appears in history; second delete on the same id returns HTTP 404 (already gone), not a silent success
Status	Pass

TC-07

Priority	High
Description	Change password with the correct current password
Reference	UC-09 Change Password
Users	Registered User
Pre-requisites	Account has a password set (not a Google-only account)
Steps	1) POST the current password and a different new password to the change-password endpoint. 2) Inspect the response. 3) Attempt to use the account's previous refresh token from another simulated session.
Input	{ currentPassword: "OldPass123", newPassword: "NewPass456" }
Expected result	HTTP 200; response includes a fresh access/refresh token pair and the updated user profile; a refresh token issued before the change is rejected as the account's tokenVersion has advanced
Status	Pass

TC-08

Priority	Medium
Description	Reject a password change when the new password equals the current one
Reference	UC-09 Change Password
Users	Registered User
Pre-requisites	Account has a password set
Steps	1) POST the change-password request with currentPassword and newPassword set to the same value. 2) Inspect the response.
Input	{ currentPassword: "SamePass123", newPassword: "SamePass123" }
Expected result	HTTP 400; response body carries code: "SAME_PASSWORD"; the stored password hash and tokenVersion are unchanged
Status	Pass

TC-09

Priority	High
Description	Sign out everywhere revokes sessions on other devices
Reference	UC-10 Sign Out Everywhere
Users	Registered User
Pre-requisites	The account is signed in on at least two simulated sessions (two valid refresh tokens)
Steps	1) From session A, POST to the logout-all endpoint. 2) From session B, attempt to use its existing refresh token to obtain a new access token.
Input	POST /api/auth/logout-all (session A's access token)
Expected result	HTTP 200 from session A; session B's subsequent refresh attempt fails as its session was revoked, forcing it to sign in again
Status	Pass

5.2 Black Box Test Cases

5.2.1 Equivalence Partitions

Video length (minutes), valid range 1-20: - Partition A (invalid, too low): length <= 0 or non-numeric -> expected: clamped to the fallback default (2) by the worker rather than crashing the pipeline. - Partition B (valid, short): 1-3 minutes -> expected: a punchy, concise script directive is used (depthForLength), fewer scenes. - Partition C (valid, mid-range): 4-12 minutes -> expected: a standard structured script directive. - Partition D (valid, long): 13-20 minutes -> expected: a depth/structure directive producing named facts and a fuller narrative structure, with sentence count scaling up to the 180-sentence ceiling rather than being capped down. - Partition E (invalid, too high): length > 20 -> expected: clamped to 20 by the worker (`Math.min(20, Math.max(1, Number(rawVideoLength) || 2))`), never silently accepted as requested.

Prompt text field: - Partition A (invalid, empty/whitespace): rejected by input validation before any provider call. - Partition B (valid, short prompt): a handful of words -> expected: normal script generation. - Partition C (valid, long prompt): near the 2000-character validation ceiling used on the image/text routes -> expected: accepted and processed. - Partition D (invalid, over-length): beyond the maxLength configured in the `validate` middleware -> expected: HTTP 400 before reaching the service layer.

5.2.2 Boundary Value Analysis (Video Length)

- length = 0: below minimum; treated as "missing" and falls back to the default of 2 minutes rather than 0 minutes of content.
- length = 1: minimum valid boundary; expected 9 sentences (per sentenceCountForVideoLength(1) = 9, confirmed by the automated test suite).
- length = 20: maximum valid boundary; expected 180 sentences, not silently reduced to what a 9-minute request would produce (this is a pinned regression test — the ceiling used to be 80 sentences, which capped every request past roughly 9 minutes).
- length = 21: above maximum; clamped down to 20 by the worker regardless of what was requested by the client or an LLM tool call.
- length = -5: negative; clamped up to the minimum by the same `Math.min(20, Math.max(1, ...))` expression.

5.2.3 Decision Table Testing

The following decision table covers the combination of conditions governing whether a video generation request proceeds: prompt present (Y/N), length within 1-20 (Y/N), daily quota remaining (Y/N).

Rule 1: prompt=Y, length=Y, quota=Y -> Action: job enqueued, HTTP 202/200. Rule 2: prompt=N, length=Y, quota=Y -> Action: HTTP 400, "Prompt is required", no job created. Rule 3: prompt=Y, length=N (out of 1-20), quota=Y -> Action: value is clamped server-side rather than rejected; job still enqueued with the clamped length. Rule 4: prompt=Y, length=Y, quota=N (limit reached) -> Action: HTTP 429, QUOTA_EXCEEDED, no job created. Rule 5: prompt=N, length=N, quota=N -> Action: HTTP 400 (validation is checked before quota, so the prompt error surfaces first).

5.2.4 State Transition Testing (Job Lifecycle)

States: queued, processing, completed, failed. - queued -> processing: triggered when a worker's atomic `findOneAndUpdate` claims the job; tested by asserting only one of two concurrent claim attempts on the same job id succeeds. - processing -> completed: triggered when the pipeline finishes without error; tested by asserting `finishJob` sets progress to 100 and stores the result payload. - processing -> failed: triggered when attempts have reached MAX_ATTEMPTS (3); tested by asserting a job on its third failed attempt is marked failed rather than re-queued. - processing -> queued (retry): triggered when a job fails with attempts remaining; tested by asserting the job's stage message reflects "Retrying (attempt N of 3)" and its lease fields are cleared. - processing -> queued (recovery): triggered on worker startup when `leaseExpiresAt` is in the past; tested by asserting `startWorker`'s recovery pass only re-queues jobs whose lease has actually expired, never a job still within its lease window (this is what stops two instances from double-processing the same render).

5.2.5 Use Case Testing

Each of the eight use cases in Chapter 3 was exercised manually end-to-end against a running instance (register through to email verification; login with both correct and incorrect credentials; a full video generation from prompt to finished playback; opening the editor, changing one scene's text, and confirming only that scene's narration audio changed; generating an image and confirming it appears in the Image Studio gallery; uploading a photo and confirming the returned suggested prompts could be sent onward into the Image Studio; a full chat conversation ending in a queued video job; and deleting/downloading items from the

Dashboard). These manual walkthroughs are not automated and are documented here as testing performed rather than claimed as scripted, repeatable test suites.

5.3 White Box Test Cases

5.3.1 Cyclomatic Complexity

Cyclomatic complexity was not measured with a static analysis tool as part of this project; it is discussed here conceptually. The functions with the highest branching complexity in the codebase are the provider fallback loops (image_generator's provider iteration in ImageGeneratorService.mjs, and chat's orderedProviders/visionProviderOrder in llm/provider.mjs), each of which has one decision point per configured provider plus error-classification branches (quota vs. invalid key vs. generic failure). These were deliberately kept as flat, sequential loops over an ordered provider list rather than nested conditionals precisely to keep their complexity linear in the number of providers rather than combinatorial, and it is exactly these functions that carry dedicated unit tests (llmProvider.test.mjs, imageProvider.test.mjs) rather than being left to integration testing alone.

The backend's automated test suite currently totals 248 passing tests across 21 test files, run with Node's built-in test runner (`node --test`). The real test files are: changePassword.controller.test.mjs, editRenderJob.test.mjs, historyPagination.test.mjs, imageAnalysis.test.mjs, imageEditJob.test.mjs, imageFailure.test.mjs, imageJobPayload.test.mjs, imageProvider.test.mjs, keywordExtraction.scoreImageRelevance.test.mjs, llmProvider.test.mjs, narrationText.test.mjs, pexelsService.safeFilename.test.mjs, rateLimit.middleware.test.mjs, security.test.mjs, skills.test.mjs, textGeneratorScript.test.mjs, validate.middleware.test.mjs, videoEffects.ffmpeg.test.mjs, videoLengthScaling.test.mjs, videoService.planClipDurations.test.mjs, and videoThumbnail.test.mjs. Together these cover narration text cleaning, video length-to-sentence-count scaling, ffmpeg effect filter correctness run against a real ffmpeg binary, image analysis JSON parsing and fallback behaviour, job payload normalisation for both image and render jobs, path-traversal and SSRF security properties, rate limiting behaviour, and history/video pagination clamping.

Honestly stated, this project prioritised backend correctness and media-pipeline reliability given the time available for a final year project, and this came at the cost of two gaps that are worth naming plainly rather than glossing over. First, there is currently zero automated

frontend test coverage — no component tests, no end-to-end browser tests. All frontend verification was manual, performed by using the application directly during development. Second, a handful of backend services have no dedicated unit tests of their own: AuthService (login/signup/token issuance logic), TTSService (the ElevenLabs/gTTS fallback path), AgentService (the chat orchestration loop), and tools.mjs (the individual tool executors) are exercised indirectly through manual end-to-end testing but not through isolated unit tests. This was a deliberate time trade-off — the media pipeline (video/image generation, effects, job queue) was judged the higher-risk, higher-complexity surface and was prioritised accordingly — and expanding both frontend test coverage and unit coverage of the untested backend services is identified as concrete future work in Chapter 9.

5.4 Performance Testing

Performance testing was informal rather than tool-driven: video generation and re-render times were observed manually across a range of video lengths (1, 5, 10, and 20 minutes) to confirm that render time scales roughly linearly with scene count rather than degrading non-linearly, and that a scene-level "Save Edit" on a single changed scene completes noticeably faster than a full re-generation of the same video, confirming the surgical re-render design actually saves time in practice and not just in theory. No dedicated load-testing tool (e.g. k6, Artillery) was used within the project timeline.

5.5 Stress Testing

Stress testing was limited to manually issuing rapid repeated requests against rate-limited endpoints (image generation, editor re-renders, chat) to confirm the in-memory rate limiter correctly returns HTTP 429 with a Retry-After header once the configured window's request count is exceeded, and that the daily quota middleware correctly blocks further generation once the day's limit is reached, using real Video/Image/Job counts rather than an easily-reset in-memory counter. Stress testing of the ffmpeg render pipeline under many simultaneous renders was not performed; the single-concurrency-per-worker-type design (Section 2.3.2, Performance) was a design decision made specifically to avoid needing to handle that scenario for the current single-process deployment target.

5.6 System Testing

System testing consisted of running the full Docker Compose stack (backend + frontend + nginx) and exercising every major user journey through the browser against the containerised build rather than the local development server, to confirm that the nginx reverse proxy

correctly forwards API, media, and SSE traffic to the backend container, that the healthcheck-gated startup order works (the frontend container does not accept traffic until the backend's /api/health probe passes), and that named volumes correctly preserve generated media across a container restart.

5.7 Regression Testing

5.7.1 Selecting Regression Tests

Every backend bug fix reviewed for this section that had a testable, deterministic root cause was converted into a permanent regression test rather than being fixed and left unverified. Confirmed examples of this include the ffmpeg zoompan filter-string crash (videoEffects.ffmpeg.test.mjs), the video-length sentence-count silent truncation bug (videoLengthScaling.test.mjs), and the path-traversal/SSRF containment properties (security.test.mjs). The selection principle applied was: if a bug could recur silently (i.e. its symptom would not be obviously visible without deliberately checking), it earns an automated regression test; if a bug was a one-off UI rendering issue caught visually, it was fixed and verified manually instead, since the project had no frontend test infrastructure to place such a test into.

5.7.2 Regression Testing Steps

1. After any change to a Service module, run the full backend suite (`npm test` in backend/, which runs `node --test "test/**/*.test.mjs"`) and confirm all 248 tests still pass before considering the change complete.

2. For changes touching the ffmpeg effect pipeline specifically, confirm videoEffects.ffmpeg.test.mjs still passes, since this file runs the actual filter strings through a real ffmpeg binary rather than only checking the JavaScript that builds them.

3. For changes touching job payload shapes or the queue worker, confirm imageJobPayload.test.mjs, editRenderJob.test.mjs, and imageEditJob.test.mjs still pass, since these pin the exact normalisation contract the workers depend on.

4. Manually re-walk the affected use case from Chapter 3 end-to-end in the running application, since frontend behaviour is not covered by the automated suite.

CHAPTER 6:
TOOLS AND TECHNIQUES

Languages

JavaScript (Node.js, ES modules): The entire backend is written as .mjs ES modules rather than CommonJS. This was chosen for native top-level await (used in a few startup paths, such as connecting to MongoDB before the server starts accepting requests) and because it is the direction the Node.js ecosystem and its tooling have moved, keeping the codebase aligned with current libraries and examples rather than mixing module systems.

JSX / JavaScript (React): The frontend is written in JSX compiled by Vite. JSX was chosen over plain JavaScript templating because the UI has genuinely stateful, interactive components (the Video Editor's scene list, the Queue panel's live-updating job cards) where React's component model and re-render behaviour are a direct fit.

Frameworks and Libraries

Express.js: Chosen as the backend web framework [4] for its minimalism — the project needed straightforward middleware composition (auth, rate limiting, quota, validation) applied selectively per route, which Express's middleware chain expresses directly without extra abstraction.

React 18 with Material-UI: React [5] was chosen for the frontend for its component reusability (the Lightbox component, for example, is shared across the Studio, Image Studio, Image Analyzer, and Dashboard). Material-UI supplied a consistent, accessible component library (dialogs, buttons, form controls) on top of which a custom design system was layered, rather than building every UI primitive from scratch.

Database

MongoDB with Mongoose: A document database [6] was chosen because the core content records (a Video with its embedded scenes array, a Job with its flexible `payload`/`result` Mixed fields) are naturally document-shaped and vary in structure between job types, which would require more schema migration overhead in a relational model. Mongoose was used on top of raw MongoDB for schema validation, indexes (the compound `(owner, createdAt)` index used by every history query), and a consistent query API across models.

Media Processing

ffmpeg / fluent-ffmpeg: ffmpeg [7] is the actual video/audio compositor — it stitches per-scene image+narration segments with transitions, burns subtitles via its libass-backed

`subtitles`/`drawtext` filters, mixes in background music, and applies visual effects (color grading, film grain, cinematic zoom via `zoompan`). fluent-ffmpeg wraps the raw command-line invocation with a more manageable JavaScript API for building filter graphs. ffmpeg was the only realistic choice for this: it is the de facto standard for programmatic video composition and is what the Docker image is built around, with a build-time check that the installed binary actually has the `subtitles` and `drawtext` filters compiled in.

Cloudinary: Used for durable storage and delivery of generated videos and images. It was chosen over storing files only on local disk because local `public/downloads`/`public/videos` directories are explicitly treated as transient scratch space (cleaned on boot, not backed up) — Cloudinary's signed delivery URLs are what make generated content survive a container rebuild and be safely shareable without exposing the underlying file path.

AI / LLM Providers

DeepSeek, Groq, Google Gemini: These three make up the text-generation fallback chain used for script writing and the chat agent (DeepSeek [11] first, then Groq [12], then Gemini [13]). The same three providers back image analysis too, through the same llm/provider.mjs module, but with a different order for vision content specifically: Groq first, then Gemini, then DeepSeek (VISION_PREFERENCE in provider.mjs), reflecting that the best-performing provider for a text completion is not necessarily the best-performing one for a vision task, so the two orderings were tuned and kept independent rather than assumed to be the same. DeepSeek and Groq were chosen as the primary text-generation choices because both expose an OpenAI-compatible `/chat/completions` API, letting one fetch-based client drive both without provider-specific SDKs; Gemini uses Google's own SDK and sits last in the text chain but second in the vision chain. Using three independent vendors, rather than one with retries, was a direct response to hitting quota and model-deprecation issues with a single provider during development — the same ordered-fallback idea informally associated with the Circuit Breaker pattern [10], though what is actually implemented here is a simpler ordered-fallback list rather than a full circuit breaker with failure thresholds and a cooldown state machine.

Text-to-Speech

ElevenLabs and Google TTS (gTTS): ElevenLabs [8] was chosen for its natural-sounding voices (two verified working voices, "Adam" and "Sarah", offered in the Studio) to give generated videos a less robotic narration than typical free TTS. Google's gTTS was kept as

the default, free fallback — narration silently drops to gTTS whenever ElevenLabs returns a 401 (invalid/expired key) or 429 (quota exhausted), so a missing or exhausted ElevenLabs key degrades the voice quality rather than breaking video generation entirely. A third `say` (system TTS) path exists as a further fallback if gTTS itself fails.

Image Generation

Flux (via Hugging Face Spaces) and Gemini: Flux, hosted through Hugging Face Spaces [14], was chosen as the primary image provider because it is an open model accessible through public Hugging Face Spaces without requiring a paid plan, giving good-quality results at no direct API cost. Gemini's [13] image model is the first fallback when Flux's hosted Spaces are unavailable or rate-limited, chosen because it was already integrated for text and vision use elsewhere in the system, reducing the number of distinct SDKs the backend depends on.

Computer Vision

Google Cloud Vision: Used as an optional enrichment step in image analysis [9] — its label-detection API supplements (rather than replaces) the vision-capable LLM's own labels, chosen because it is a mature, purpose-built label-detection service that is more reliable for pure object/tag detection than asking a general LLM to enumerate every visible object exhaustively.

Stock Photos

Pexels API: Used to source real stock photography [15] as an alternative to AI-generated images for a video's visuals, and as a tool the chat agent can call directly (search_stock_images). Pexels was chosen for its free, well-documented API and permissive usage terms suitable for a project without a commercial stock-photo licensing budget.

Authentication and Security

JWT (jsonwebtoken) and bcrypt (bcryptjs): JWTs [3] were chosen for stateless session authentication so the API does not need a server-side session store, with a `tokenVersion` claim added specifically to make sessions revocable (logout-all, password change) despite JWTs being stateless by default. bcrypt was chosen for password and OTP hashing as the established, battle-tested standard for this purpose, with a work factor of 10 applied consistently across both use sites (account passwords and email OTP codes).

DevOps

Docker and Docker Compose: The whole stack — backend, frontend, and an optional local MongoDB — is containerised so the deployment environment matches development exactly, including the specific ffmpeg build with libass support that the video pipeline depends on. Docker Compose was chosen over a more complex orchestrator because the project's actual deployment target is a single host running a small, fixed set of services, for which Compose's declarative multi-container file is sufficient without the operational overhead of Kubernetes.

nginx: Used in the frontend container to serve the built Vite bundle as static files and reverse-proxy API/media/SSE requests to the backend container, so the browser only ever talks to one origin in production. This was chosen specifically to keep the frontend's API client using a simple relative base URL rather than needing environment-specific absolute URLs baked into the bundle at build time.

Testing

Node.js built-in test runner (`node --test`): Chosen over an external framework (Jest, Mocha) specifically to avoid adding a test-framework dependency to a backend that otherwise has none, since Node 18+'s built-in runner already provides everything the project's test suite needs (describe/test blocks, node:assert/strict, before hooks) without extra tooling.

Version Control

Git and GitHub: Used for source control and collaboration throughout development, with a Git worktree-based workflow used for isolated feature branches during later stages of the project.

CHAPTER 7:
SUMMARY AND CONCLUSION

This project set out to build a single platform that could take a short text prompt and produce a complete narrated video — script, visuals, voice, and composed output — while also being editable afterwards at the level of individual scenes, rather than requiring a full regeneration for every small change. That core goal was met: FRAME Studio generates videos from one to twenty minutes long with scaling script depth, supports four different visual sources (AI-generated images, stock photography, local uploads, or a mix), narrates them with a choice of voices, and lets a user return to a finished video and re-render only the scenes or effects they actually changed. Around that core, the project also delivered a standalone Image Studio, an Image Analyzer with real label detection and description generation, and an agentic chat mode that can drive all of these capabilities conversationally after clarifying the details it needs.

What worked well was the decision, made fairly early, to treat unreliable third-party APIs as the normal case rather than the exception. Every generation capability — text, TTS, and images — ended up with its own explicit, ordered fallback chain, and this paid off directly during development: providers were hit with quota limits, deprecated models, and outright outages multiple times, and in every case the system degraded to a working (if occasionally lower-quality) result instead of failing the whole feature. The background job queue, redesigned around atomic lease-based claiming rather than a simple in-memory "is a job running" flag, was similarly a design investment that turned out to matter in practice once the failure mode it was built to prevent — a server restart mid-render causing a duplicate render — was actually observed and fixed.

The project is also honest about what did not get finished to the same standard. Frontend automated testing does not exist; every frontend behaviour was verified manually. Several backend services — authentication, TTS, the chat agent's orchestration loop, and the individual tool executors — do not have dedicated unit tests, even though they are exercised indirectly through manual end-to-end use. This was a conscious trade-off given the project's timeline: the media pipeline and the security-sensitive paths (path traversal, SSRF, rate limiting, quota enforcement) were judged to be the highest-risk, highest-complexity surfaces and were prioritised for automated coverage accordingly, at the cost of leaving other real areas of the system less rigorously verified than they ideally would be.

Security was not treated as a checklist item added at the end but as something discovered and fixed iteratively while building the media pipeline: generated media was originally served

from open static file mounts with no access control at all, which was closed by moving all media delivery behind authenticated, short-lived tokens; file upload and deletion paths were vulnerable to path traversal until explicit containment checks were added; and the video effects pipeline's remote-URL handling needed an SSRF allowlist once it became possible to point a re-render at an arbitrary URL. None of these were requirements specified up front — they were found by building the system honestly and then asking what could go wrong with what had just been built.

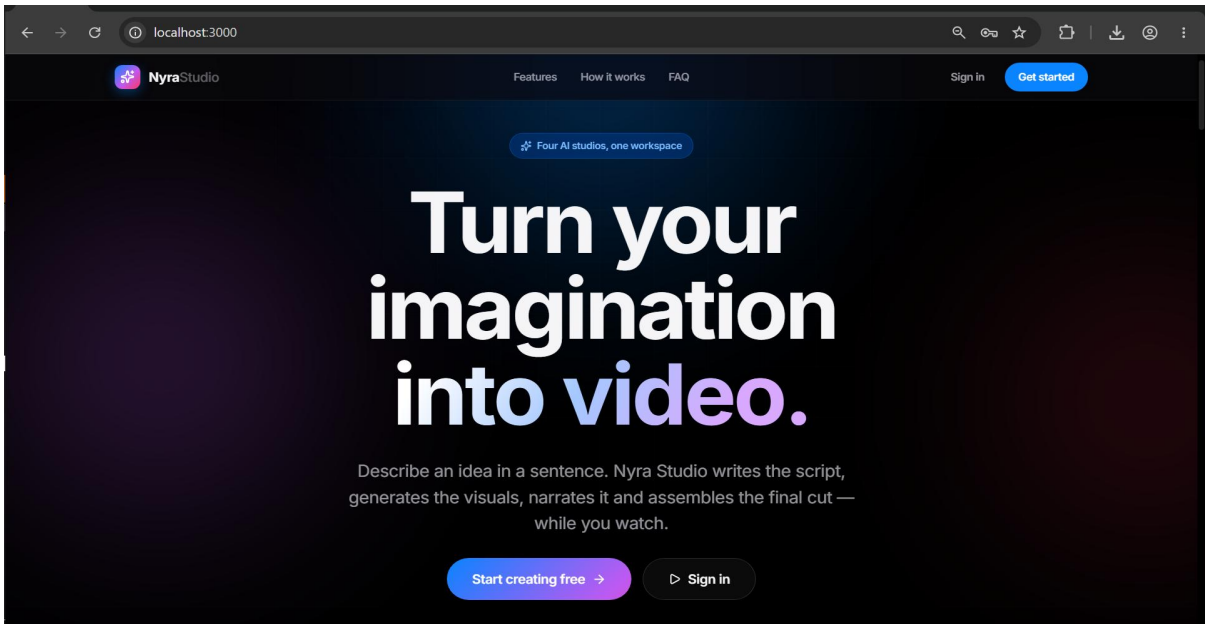
Overall, the project demonstrates a working, non-trivial full-stack AI application with real engineering depth in the places that mattered most for its core promise — reliable generation despite unreliable dependencies, and genuinely surgical editing rather than "regenerate everything" editing — while being transparent about the areas, principally frontend testing and coverage of a handful of backend services, that were consciously left for a future iteration rather than rushed to a lower standard under time pressure.

CHAPTER 8:
USER MANUAL

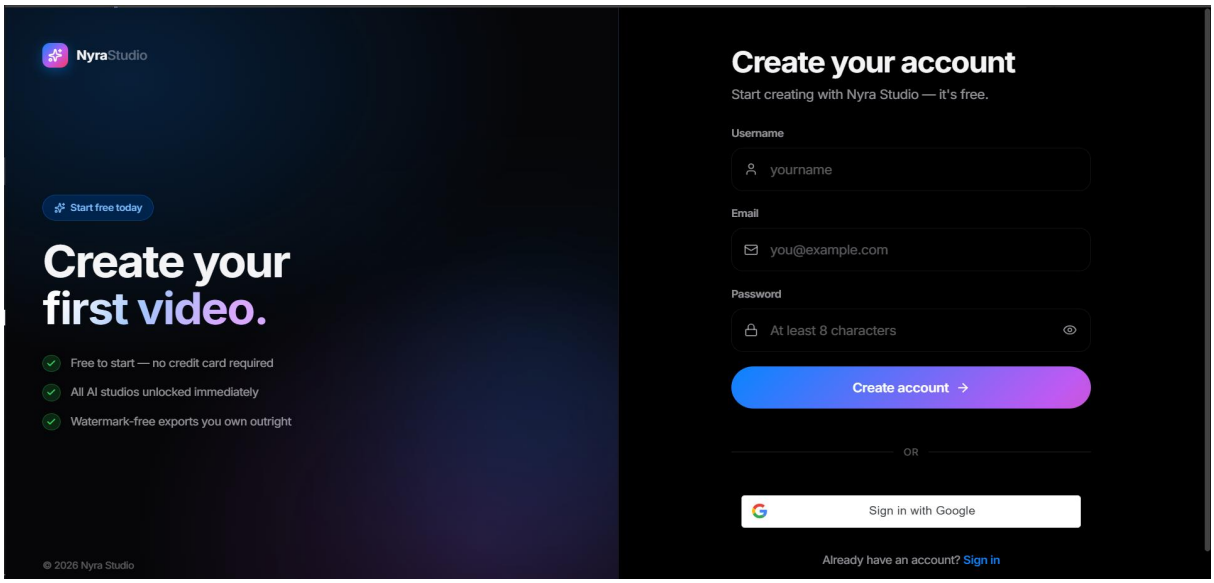
The live deployment used for the screenshots and walkthroughs in this chapter is reachable directly: the frontend is hosted at https://fyp-project-sand.vercel.app/ (Vercel), and it talks to the backend API hosted at https://fypproject-production-7f42.up.railway.app/ (Railway), whose health check is visible at /api/health. A reader may sign up and follow every step below against this running instance rather than only reading about it.

8.1 Sign Up / Login

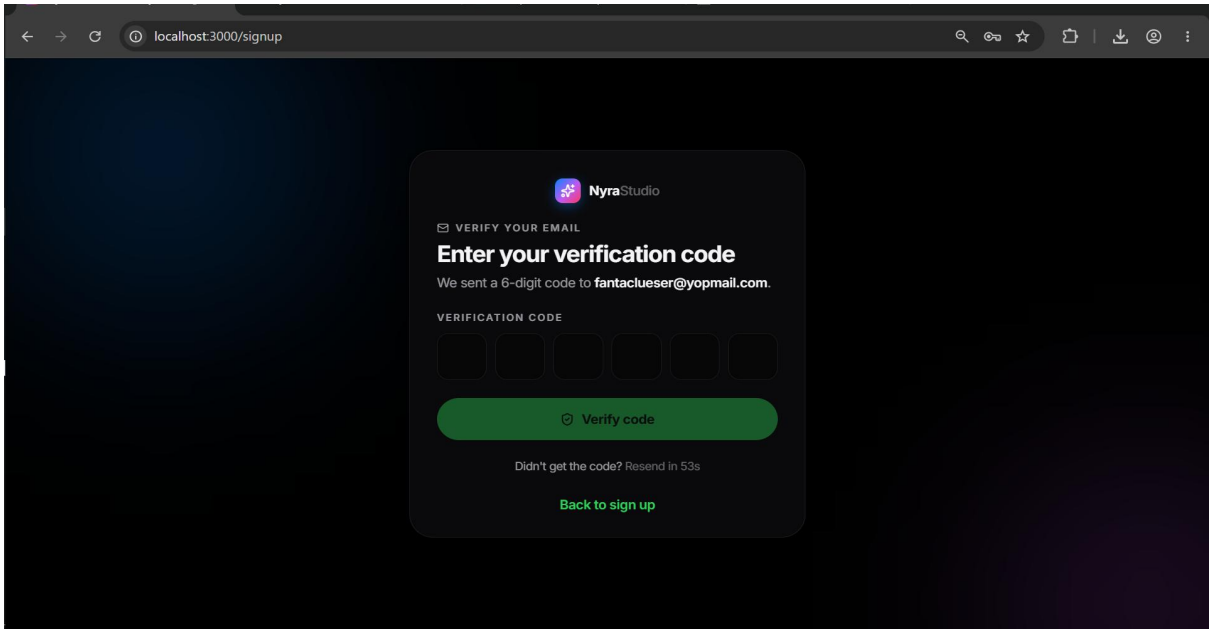
1. Navigate to the landing page and click "Sign Up".



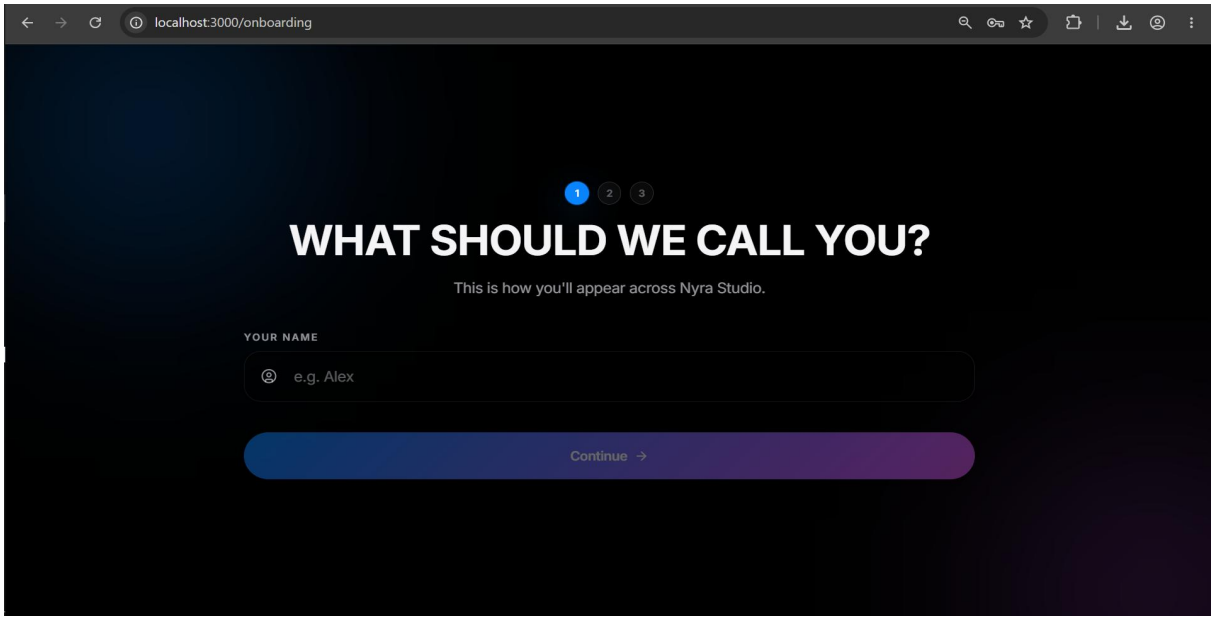
2. Enter a username, email address, and password, then submit the form.

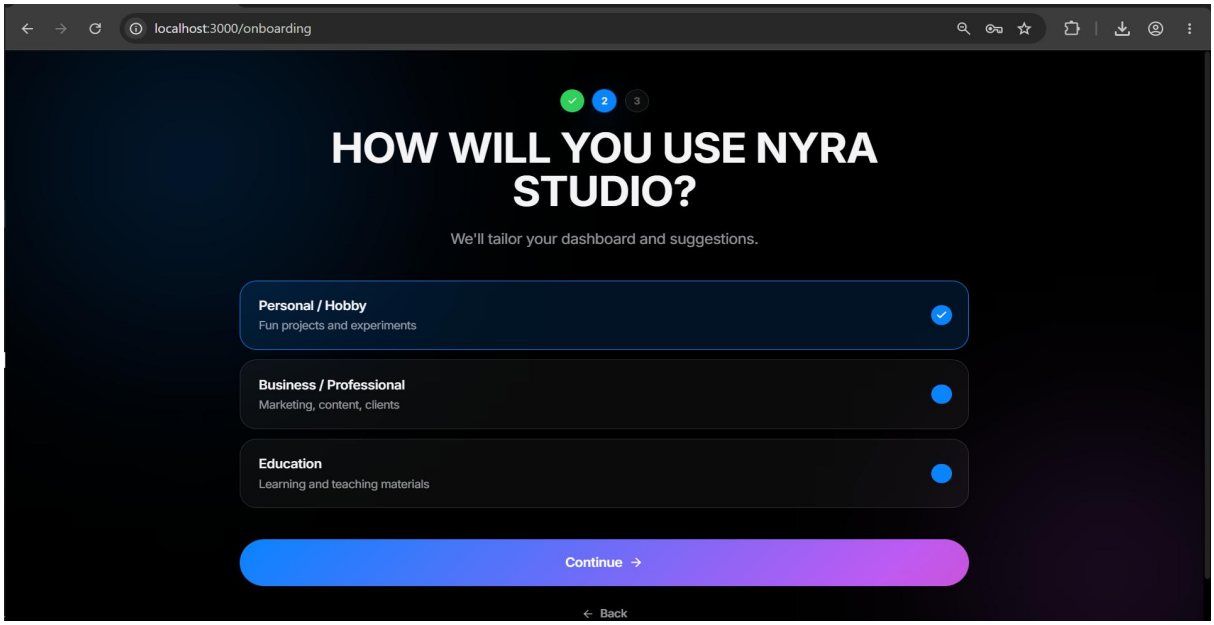


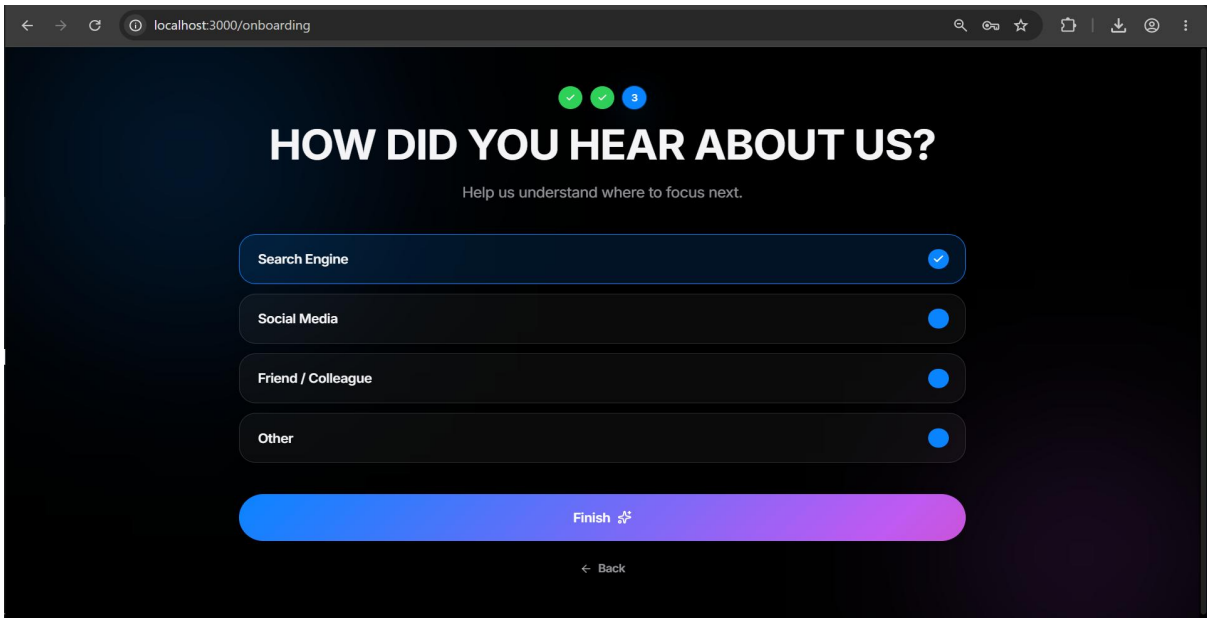
3. Check the inbox for the given email address for a 6-digit verification code and enter it on the verification screen. If the code does not arrive, use "Resend Code".



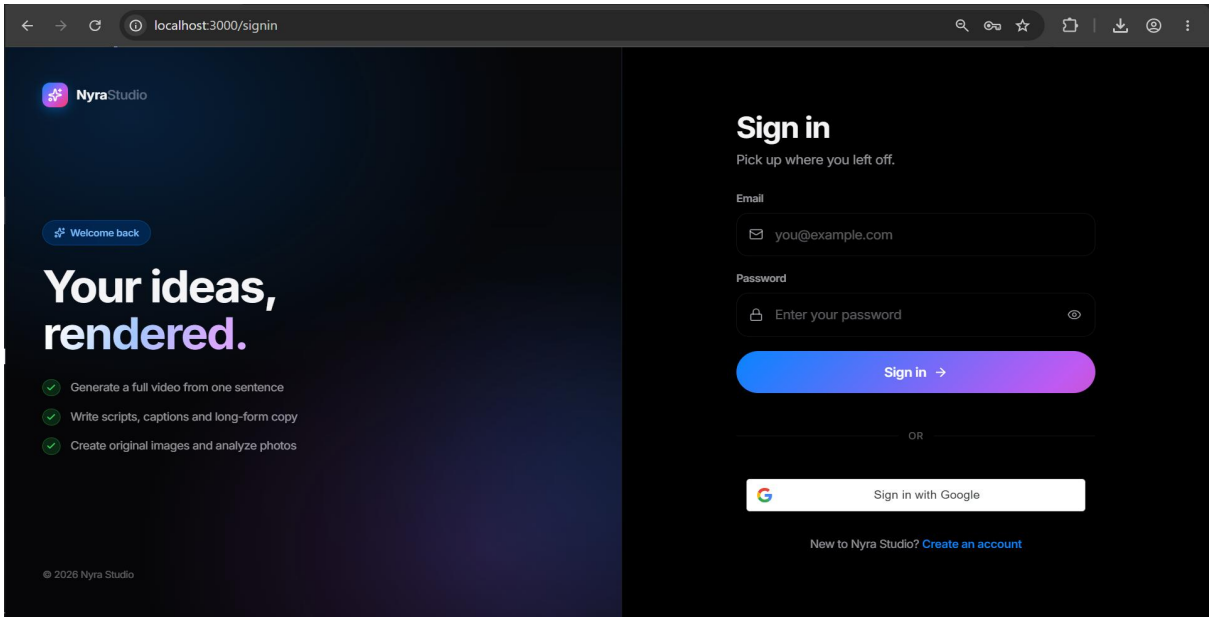
4. Once verified, you are signed in automatically and redirected to a short, three-question onboarding flow before landing on the Dashboard: your display name, how you plan to use FRAME Studio (personal, business, or education, which tailors the dashboard and suggestions), and how you heard about the platform.



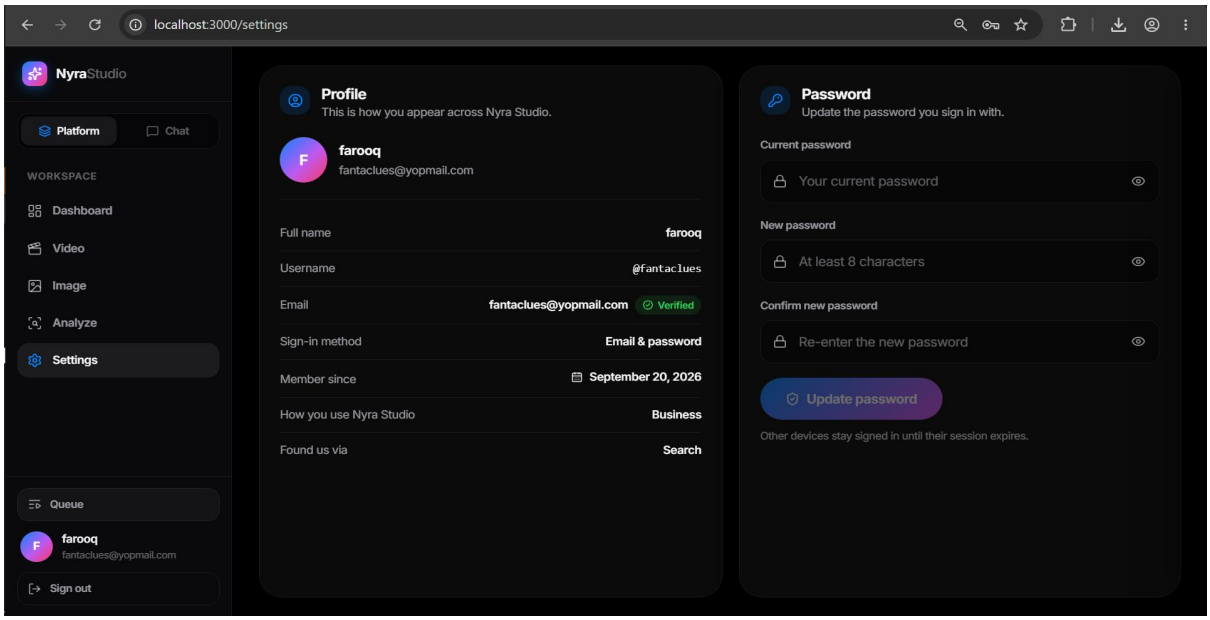




5. To sign in again later, use the Login screen with the same email and password, or "Sign in with Google" if the account was created that way.

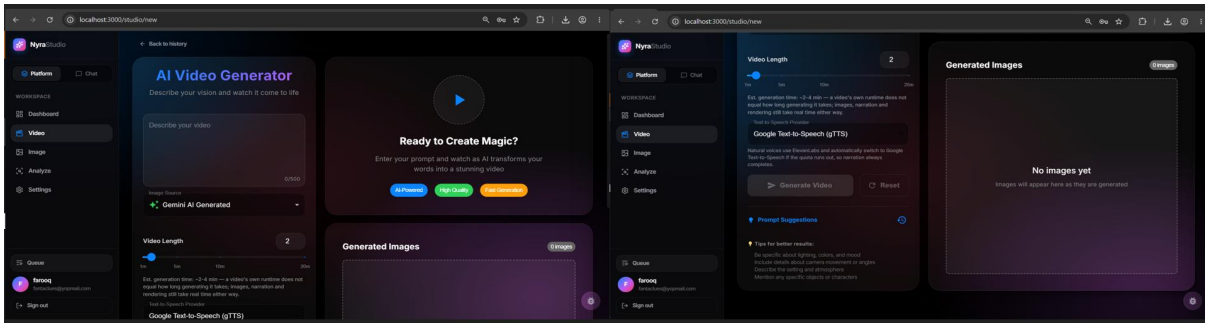


6. From Settings, you can change your password or sign out of every device at once ("Sign out everywhere"), which immediately invalidates any other active session.



8.2 Generate a Video from a Prompt

1. Open the Studio from the main navigation.



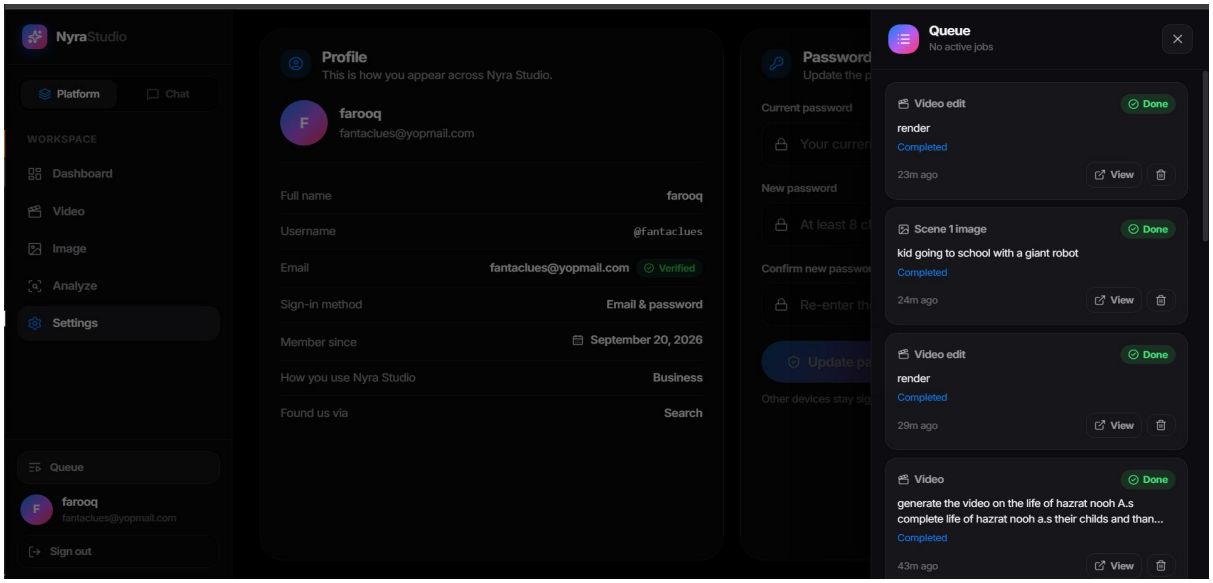
2. Enter a topic or prompt describing the video you want.

3. Set the video length using the length control, anywhere from 1 to 20 minutes. Shorter lengths produce a punchier, more concise script; longer lengths produce a fuller script with more named facts and structure.

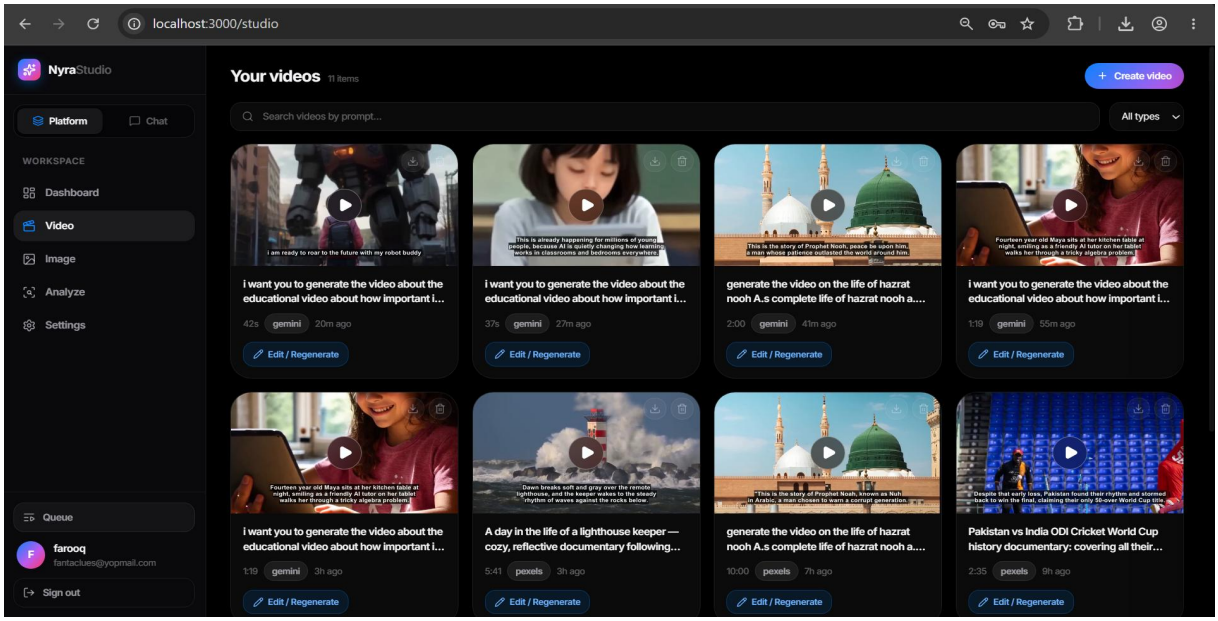
4. Choose the image source: AI-generated (labelled "Gemini AI Generated" in this menu, since that option is what selects the AI-image path — the actual provider tried first behind it is Flux, with Gemini and further fallbacks used only if Flux is unavailable, as described in Chapter 6), stock photos (Pexels), your own uploaded images, or a mix.

5. Choose a narration voice (Google TTS by default, or one of the ElevenLabs natural voices if configured).

6. Click Generate. The request is queued as a background job; you can navigate away and check progress later from the Queue panel.



7. Once the job completes, the finished video appears in your Studio library.



8.3 Edit a Video

1. From the Studio library or Dashboard, open a previously generated video in the Video Editor.

2. Use the Scenes tab to review each scene's image, narration text, and duration. Edit a scene's subtitle text to have it re-narrated, or click a scene to replace its image (choose "Generate with AI" for a new AI image, or select from stock/uploads).

3. To remove a scene, use the scene's remove option; to add a new scene, use the add-scene control and provide its narration text and image.

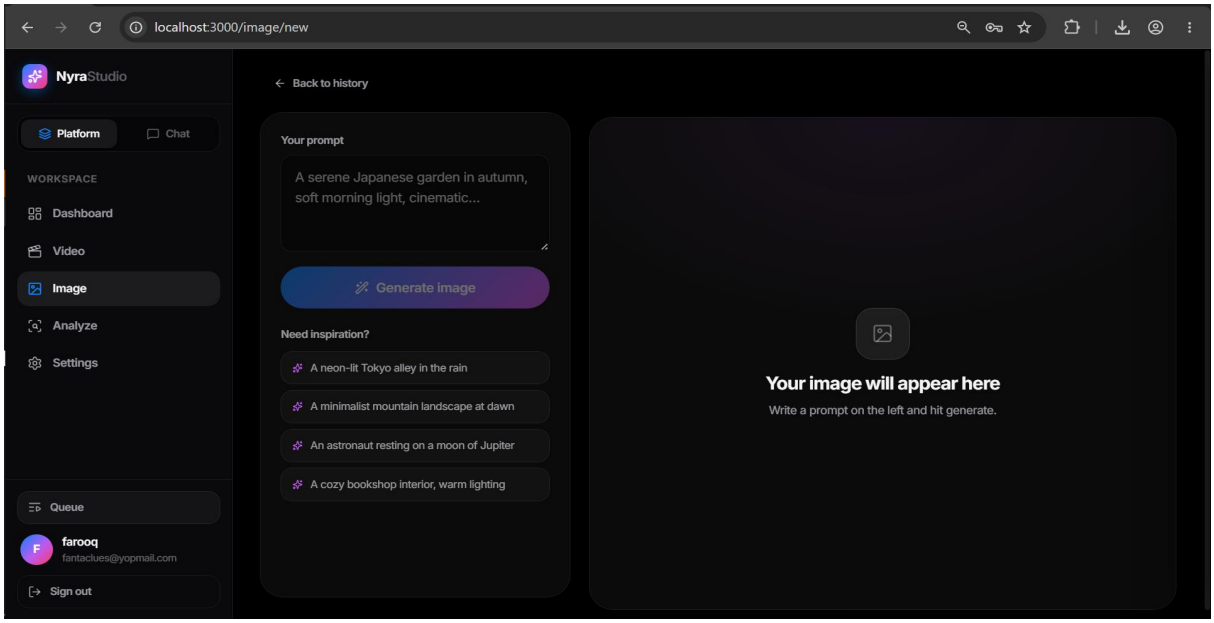
4. Switch to the Effects tab to apply visual effects such as cinematic zoom, color grading, film grain, or tone adjustments. Multiple effects can be combined.

5. Click "Save Edit" (for scene changes) or "Apply Effects" (for visual effects). The re-render is queued as a background job and reported in the Queue panel; only the scenes you actually changed are re-narrated, so this is typically much faster than generating a new video from scratch.

6. Once complete, the edited version appears as a new entry in your library; the original video is not overwritten.

8.4 Generate an Image

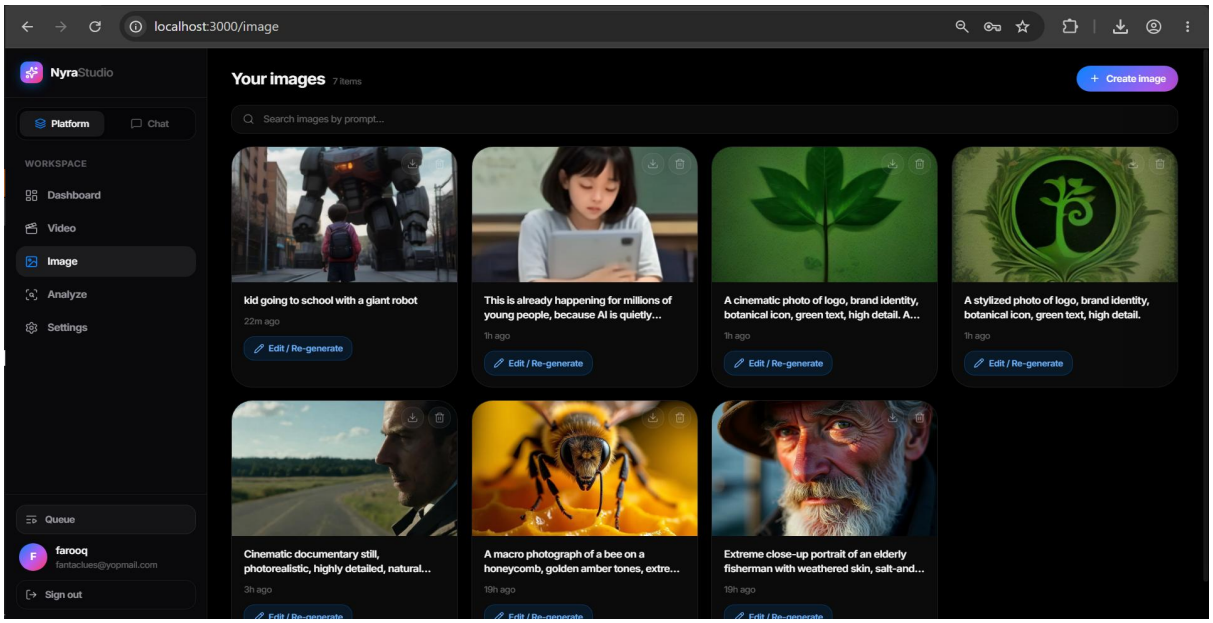
1. Open the Image Studio.



2. Enter a text prompt describing the image you want.

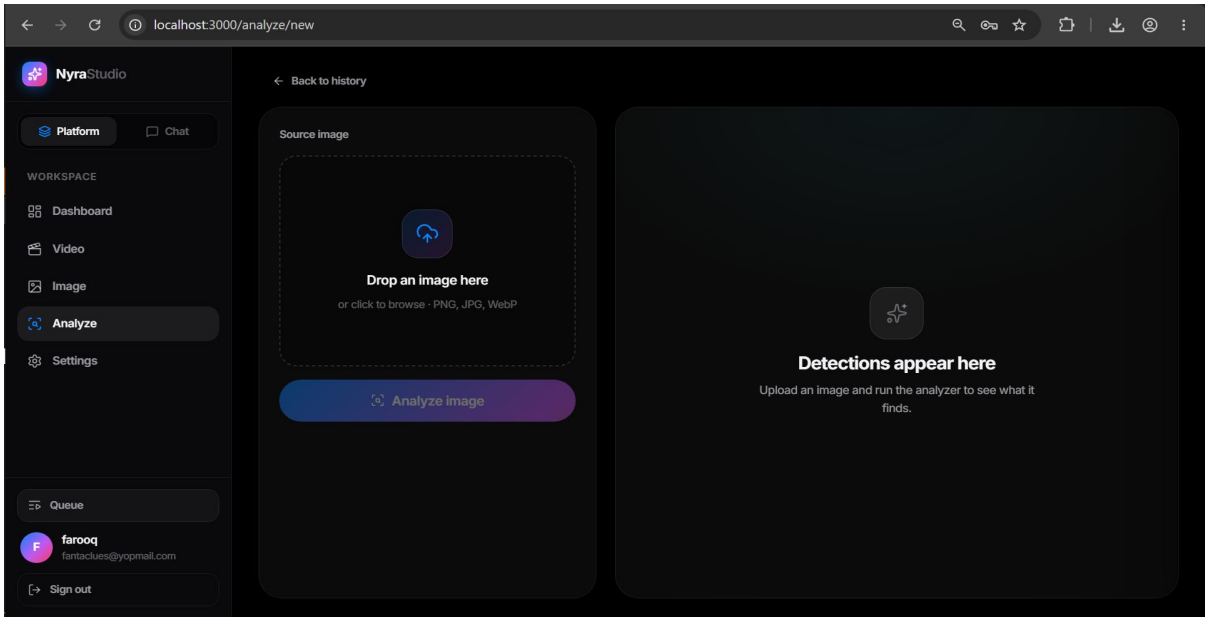
3. Click Generate. The image pipeline tries its configured providers in order and returns the result, typically within a short wait (no queue step is required for a single image from this page).

4. The generated image appears in your gallery and can be viewed full-size, downloaded, or used again with a rewritten prompt via "Save Edit".



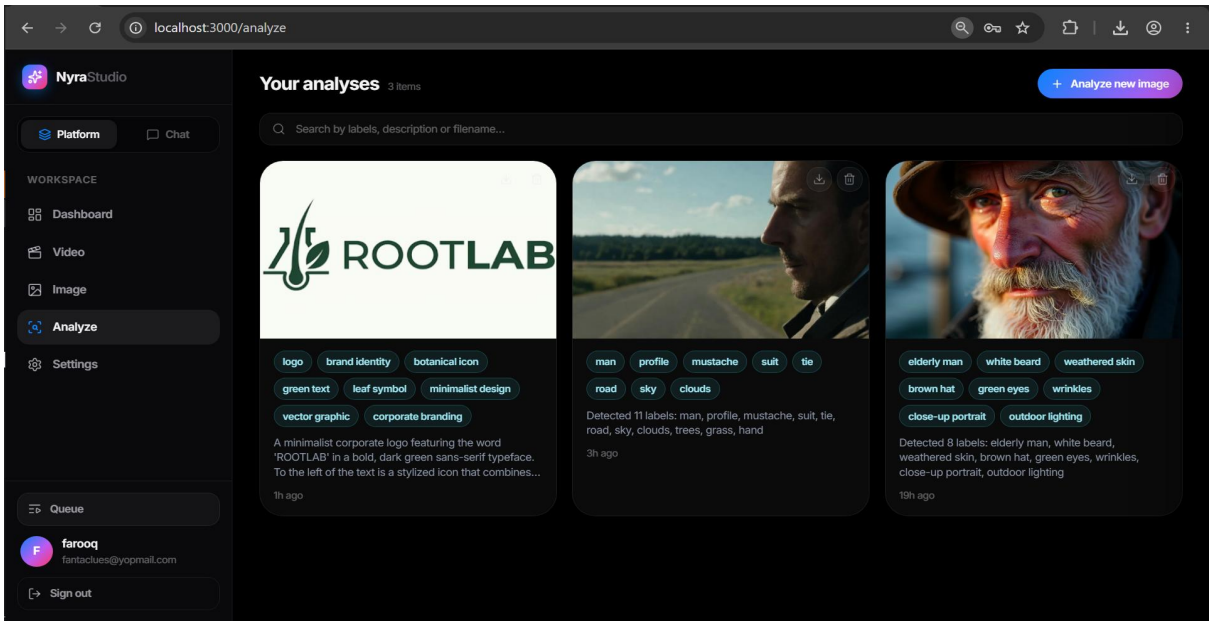
8.5 Analyze an Image

1. Open the Image Analyzer.



2. Upload a photo from your device.

3. Wait for the analysis to complete. The result shows a list of detected labels, a written description of the photo, and three suggested prompts, each offering a different style or angle for recreating or riffing on the image.

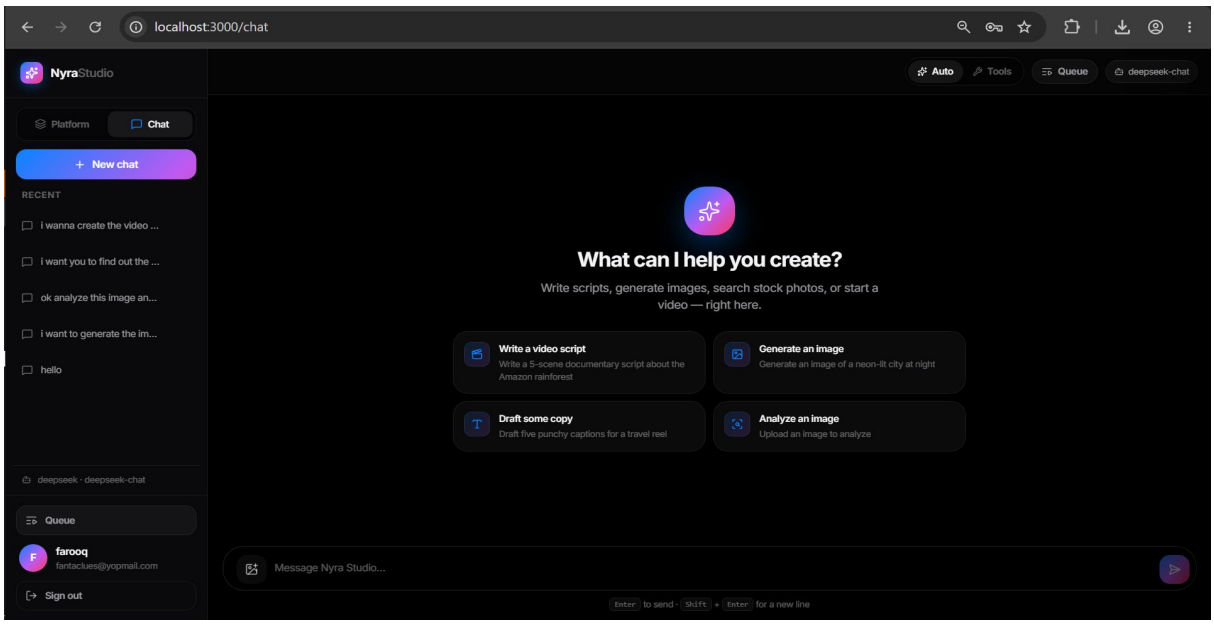


4. Click "Use this prompt" on any suggested prompt to send it directly to the Image Studio or Video Studio to generate new content from it.

5. Use "Export PDF" to download a formatted report of the analysis, including the image, labels, and description.

8.6 Use the Agentic Chat

1. Open the Chat page.



2. Select a capability mode if you want to restrict the assistant to one task (Script Writer, Image, Video, Analyzer, or Q&A), or leave it on Auto to let it decide.

3. Describe what you want in plain language, for example "make me a 5-minute video about the history of coffee using AI images."

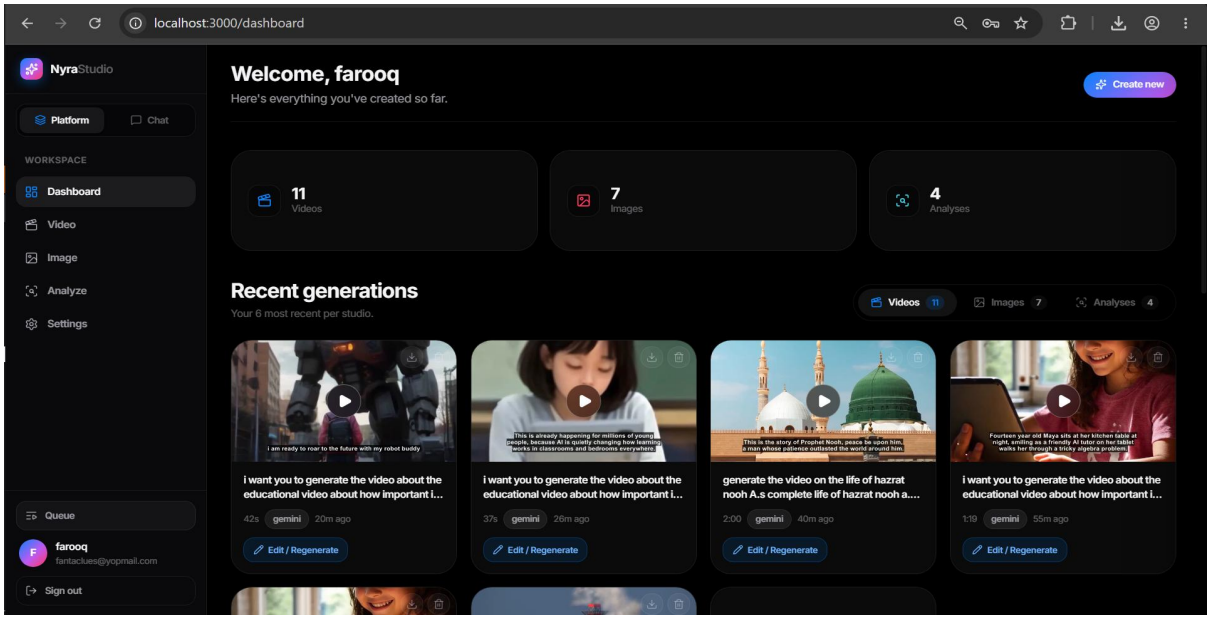
4. The assistant will ask clarifying questions if needed (length, visual style) before taking any action, and will restate the plan for you to confirm.

5. Once you confirm, the assistant calls the relevant tool (for example, queuing a video generation job) and reports back with a summary and a link to track progress in the Queue.

6. Past conversations are saved and can be reopened from the conversation list; deleting a conversation removes its message history.

8.7 View and Manage Your Dashboard/History

1. Open the Dashboard to see a unified view of your generated videos, images, and analyses.



2. Use the search box to filter items, and the pagination controls to move between pages of results.

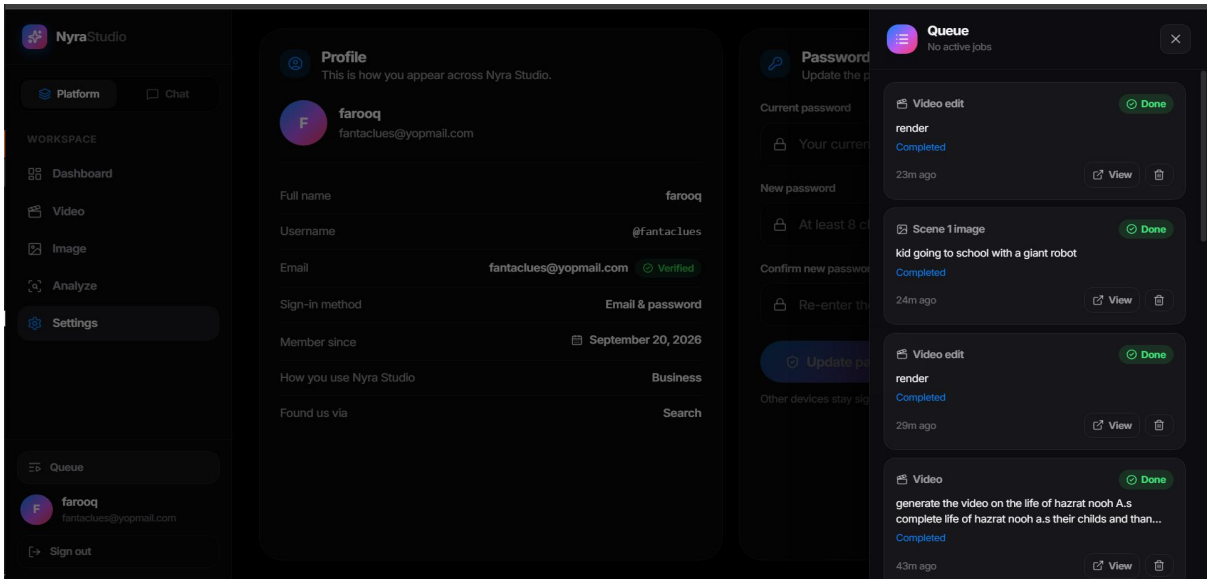
3. Click any item to open it in the Lightbox viewer for a larger preview.

4. Use the Download button on an item to save it to your device.

5. Use the Delete button to permanently remove an item; you will be asked to confirm, since this cannot be undone.

8.8 Monitor the Background Queue Panel

1. Open the Queue panel from the main navigation (or from within the Studio/Editor where an action was just queued).



2. Each entry shows the job type (video, image, or render), its current status (queued, processing, completed, or failed), a progress percentage, and a short stage description (for example, "Composing segment 3 of 12").

3. A completed job shows a link to view the finished result; a failed job shows the reason it failed where available.

4. You can remove a job entry from the panel once it is no longer needed; this does not delete any content that job already produced.

CHAPTER 9:
LESSONS LEARNT AND FUTURE WORK

9.1 Lessons Learnt

Filter strings are not just JavaScript strings. The most instructive bug of the project was in the video effects pipeline: combining several ffmpeg filters (Documentary Style, Cinematic, Color Grading, Film Grain, and tone adjustments) crashed ffmpeg outright with an unsigned integer wraparound exit code, which looked like a low-level ffmpeg fault. The actual cause was that filter expressions like `zoompan`'s `z=`/`x=`/`y=` parameters and `curves=all=` had been wrapped in double quotes in the JavaScript template strings that built them, which is valid-looking JavaScript but invalid syntax for ffmpeg's own internal expression evaluator. No amount of checking that the JavaScript was well-formed would have caught this, because the bug only exists once ffmpeg itself tries to parse the string. The fix, and more importantly the lesson, was that any code generating a string in someone else's domain-specific language has to be tested by actually running it through that language's own parser — which is why videoEffects.ffmpeg.test.mjs runs every effect combination through a real ffmpeg binary rather than only asserting on the JavaScript.

Fallback chains have to be designed, not bolted on. The first version of image and TTS generation called a single provider and let the whole feature fail if that provider was unavailable. Adding fallback later meant going back through each service and deciding, provider by provider, what "this provider is unavailable" actually looks like in its API responses (a 401 versus a 429 versus a plain network error mean different things and sometimes warrant different fallback behaviour), which was considerably more work than it would have been to design the ordered-provider-list pattern in from the start. The lesson carried into later services (the LLM provider layer was built with this pattern from day one) was to treat "which provider handled this, and what happens when it can't" as a first-class design question for any external API integration, not an afterthought.

A silent cap is worse than an explicit error. The video-length-to-sentence-count function had a hard ceiling of 80 sentences that predated the UI's 1-20 minute length control being extended to its full range. A 20-minute request silently produced the same script length as a roughly 9-minute request, with only a console warning that no user would ever see — the video "worked" but was quietly wrong. This was found by comparing actual output durations against requested lengths rather than just checking that generation succeeded, and the fix (and the regression test that now pins it) reinforced that "it didn't crash" and "it did what was asked" are different claims that both need checking.

CSS stacking contexts do not respect intent. The shared Lightbox component (used for full-size media viewing across the Studio, Image Studio, Image Analyzer, and Dashboard) was originally rendered inline within the page's component tree with `position: fixed`. In practice, an ancestor element with an active CSS transform or animation establishes its own containing block for fixed-position descendants, which meant the lightbox could end up trapped behind the app's sidebar or clipped by a scrolled container in certain page states, despite `position: fixed` normally being expected to escape any ancestor. The fix was to render the Lightbox through a React portal directly onto `document.body`, which sidesteps the containing-block problem entirely rather than trying to chase down every ancestor that might one day gain a transform. The lesson was that "fixed" is not actually a global guarantee in CSS, and a component meant to always appear on top of everything else is more robust rendered outside the normal component tree than fought into position within it.

Pagination defaults matter more than they seem to. An early version of the history/dashboard list endpoints had no upper bound on the requested page size, so a client (or a bug in the client) asking for a very large `limit` would have the backend attempt to return the entire collection in one response. This was caught and fixed by clamping the limit server-side (to 100) regardless of what was requested, with the clamping behaviour itself pinned by historyPagination.test.mjs. The general lesson — never trust a client-supplied bound without a server-side ceiling — applied again almost immediately afterward to the effects array (capped at 20) and to validated array fields more generally in the `validate` middleware.

9.2 Future Work

Frontend automated testing. The most significant gap identified in Chapter 5 is the complete absence of frontend test coverage. Introducing component-level tests (React Testing Library) for the Video Editor's scene management and the Queue panel's polling/state logic, and a small set of end-to-end browser tests (Playwright) for the core generation flows, would close this gap and is the highest-priority item for continued work.

Unit tests for the remaining backend services. AuthService, TTSService, AgentService, and the individual tool executors in tools.mjs currently rely on manual verification. Extracting their pure logic (token issuance rules, OTP expiry/attempt logic, the agent's tool-call execution loop) into testable functions, following the same pattern already used successfully elsewhere in the codebase, would extend the existing 248-test suite to cover these paths.

A CI/CD pipeline. The project currently runs its test suite manually. Wiring the existing `npm test` command into a GitHub Actions workflow that runs on every pull request, and extending it to build and smoke-test the Docker images, would catch regressions before they reach a deployed environment rather than relying on a developer remembering to run tests locally.

Multi-language support. Both the narration pipeline (gTTS already supports a dozen languages) and the UI currently assume English. Extending the script-generation prompt and TTS provider selection to accept a target language, and adding i18n to the React frontend, would make the platform usable for a meaningfully larger audience without requiring architectural changes to the pipeline itself.

Parallelising per-scene generation. The current video pipeline generates each scene's image and narration sequentially in a loop. As the Activity Diagram in Chapter 4 notes, image generation and TTS synthesis for a given scene do not depend on each other and could run concurrently, and independent scenes could be processed in parallel batches rather than strictly one after another, which would meaningfully reduce total generation time for longer videos.

Horizontal scaling of the job workers. The lease-based job-claiming design in JobService.mjs was deliberately built to be safe across multiple concurrent worker processes even though the current deployment runs a single backend instance. Actually running multiple backend replicas behind a load balancer, with the existing lease/heartbeat mechanism handling job distribution between them, would let the render pipeline scale horizontally under real load without further changes to the queue logic itself.

A native mobile application. The current platform is web-only. A companion mobile app (or a installable PWA as a lighter-weight first step) would let users track queued generation jobs and browse their history without keeping a browser tab open, which is a natural extension given how much of the current UX already revolves around background jobs completing while the user is doing something else.

An administrative role. The User model already reserves an 'admin' role that is not yet exercised anywhere in the application. Building out admin-facing views (aggregate quota/usage monitoring, the ability to investigate a failed job across all users, moderation of

generated content) would be a natural next step for operating the platform beyond a single-developer deployment.

REFERENCES

[1] Sommerville, I., "Software Engineering," 10th ed., Pearson Education, 2015.

[2] International Organization for Standardization, "ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," ISO/IEC, Geneva, Switzerland, 2011. Available: https://www.iso.org/standard/35733.html

[3] Jones, M. B., Bradley, J., and Sakimura, N., "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. Available: https://www.rfc-editor.org/rfc/rfc7519

[4] OpenJS Foundation, "Express.js Documentation," 2024. Available: https://expressjs.com/

[5] Meta Platforms, Inc., "React Documentation," 2024. Available: https://react.dev/

[6] MongoDB, Inc., "MongoDB Manual," 2024. Available: https://www.mongodb.com/docs/manual/

[7] FFmpeg Developers, "FFmpeg Documentation," 2024. Available: https://ffmpeg.org/documentation.html

[8] ElevenLabs, "ElevenLabs API Reference," 2024. Available: https://elevenlabs.io/docs/api-reference

[9] Google Cloud, "Cloud Vision API Documentation," 2024. Available: https://cloud.google.com/vision/docs

[10] Fowler, M., "Circuit Breaker," martinfowler.com, 2014. Available: https://martinfowler.com/bliki/CircuitBreaker.html

[11] DeepSeek, "DeepSeek API Documentation," 2024. Available: https://api-docs.deepseek.com/

[12] Groq, "GroqCloud API Documentation," 2024. Available: https://console.groq.com/docs

[13] Google, "Gemini API Documentation," 2024. Available: https://ai.google.dev/gemini-api/docs

[14] Hugging Face, "Spaces Documentation," 2024. Available: https://huggingface.co/docs/hub/spaces

[15] Pexels, "Pexels API Documentation," 2024. Available: https://www.pexels.com/api/documentation/

APPENDIX A:
ENVIRONMENT SETUP AND INSTALLATION GUIDE

A.1 Overview

This appendix documents how to actually run FRAME Studio from source, for a reader who wants to reproduce the system rather than only read about it. Two supported paths exist for running the project locally, and they are deliberately kept independent: running the backend and frontend directly with Node for day-to-day development with hot reload, or running the whole stack with Docker Compose as a single reproducible unit. Both paths read the same backend/.env file, so no configuration is duplicated between them. The Docker path is also the deployment target this project is built towards for a self-managed VM or a future migration off managed hosting; the live deployment actually running today, described in A.8, uses that same Dockerfile on a managed platform rather than a hand-run VM, and does not replace or change anything documented in A.3-A.6.

A.2 Prerequisites

Option A (local Node) requires Node.js 22 and a working ffmpeg/ffprobe installation reachable on the system PATH, since the video pipeline shells out to the real ffmpeg binary rather than a bundled library. Option B (Docker) requires only Docker Desktop on Windows/macOS, or Docker Engine with Compose v2 on Linux, confirmed by docker compose version printing a v2.x release; the backend image installs its own ffmpeg build with libass and freetype support baked in, so no host-level ffmpeg install is needed for that path. Both paths need a MongoDB connection string (MongoDB Atlas in the deployed configuration, or a local Mongo container under Docker's optional local-db profile), and API keys for at least one text generation provider, one TTS provider, and Cloudinary for durable media storage; every other provider key is optional and the corresponding fallback chain simply skips a provider whose key is absent.

A.3 Option A — Local Node (development)

Install dependencies for both halves of the stack independently, since the backend and frontend are separate npm projects with their own package.json and lockfile:

1. cd backend && npm install
2. cd ../frontend && npm install

Configure the environment by copying backend/.env.example to backend/.env and filling in the required keys (MongoDB URI, JWT secret, at least one LLM provider key, a Pexels key, and Cloudinary credentials). The application deliberately fails fast at startup rather than

silently limping along with missing configuration: assertRequiredEnv() in config/env.mjs throws immediately in production if a required variable is absent, which surfaces a misconfiguration the moment the server starts rather than several requests later as a confusing runtime error.

Run the two halves in separate terminals:

backend: cd backend && npm start (serves the API on http://localhost:5000) frontend: cd frontend && npm start (Vite dev server on http://localhost:3000, which proxies /api requests to the backend automatically so the frontend code can use relative URLs identically in development and production)

Run the automated backend test suite with cd backend && npm test, which exercises the narration/video timing algorithm, the ffmpeg filter-chain builders, the topic-anchored image relevance scorer, and the request-validation and rate-limiting middleware described in Chapter 5.

A.4 Option B — Docker Compose (whole stack)

The whole stack — the Express/ffmpeg backend, the built React SPA served through nginx, and optionally a local MongoDB container — is brought up with a single command once the root .env.docker.example has been copied to .env:

cp .env.docker.example .env docker compose up -d --build

The first build installs ffmpeg and both projects' npm dependencies inside their respective images, so it takes a few minutes; subsequent builds are cached and take seconds. Once running, the application is reachable at http://localhost:8080 and the API health check at http://localhost:5000/api/health; docker compose logs -f backend shows render progress and ffmpeg output live. backend/.env is injected into the container at runtime through Compose's env_file mechanism rather than being copied into the image at build time, so the same credentials work unmodified across both run modes and are never baked into a distributable image layer.

Named Docker volumes back every path the pipeline writes generated media to (public/downloads, public/videos, public/uploads, public/segments, public/audio/scenes, public/audio/tts-cache), so a container rebuild or docker compose down does not discard in-

progress work; only docker compose down -v deletes those volumes, and is used deliberately rather than by accident.

A.5 Common operational commands

docker compose ps: container and healthcheck status for every service. docker compose logs -f backend: follow render progress and ffmpeg output. docker compose restart backend: restart only the API without rebuilding. docker compose exec backend npm test: run the backend's automated test suite inside the running container, against the exact environment the container provides. docker compose down: stop and remove containers while keeping the named volumes (generated media survives). docker compose down -v: stop and remove containers and volumes together (destructive; deletes any media not yet uploaded to Cloudinary).

A.6 Known setup caveats

A few details are easy to trip over the first time and are documented here plainly rather than left to be rediscovered by trial and error. Port 5000 is used by both run modes' backend, so running Option A and Option B at the same time fails with "port is already allocated" — stop one before starting the other, or set BACKEND_PORT to a different value in the root .env for the Docker stack. NODE_ENV=production (which Compose sets by default) changes one visible behaviour: the signup OTP response no longer echoes the code in the API response body when an email fails to send, since that convenience is only safe in a non-production environment; the code is still written to the backend log. Google sign-in is compiled into the frontend bundle at build time through VITE_GOOGLE_CLIENT_ID, so changing it requires rebuilding the frontend image rather than only restarting the container, and an empty value is handled gracefully by disabling the Google sign-in button rather than crashing the page.

A.7 Key backend API routes referenced in this document

A short index of the routes exercised by the use cases in Chapter 3 and the test cases in Chapter 5, for a reader cross-referencing this appendix against the running application: POST /api/auth/signup and /api/auth/login for account creation and authentication; POST /api/video/generate and GET /api/jobs/:id for video generation and job polling; POST /api/video/editor/queue-re-render for scene-level re-editing; POST /api/image/generate for standalone image generation; POST /api/image/analyze for the Image Analyzer; the chat routes under /api/chat for the agentic assistant; and GET /api/health, used by the Docker healthcheck and referenced in Chapter 4's deployment diagram discussion.

A.8 Option C — Live deployment (Vercel + Railway)

Alongside the two local run modes above, the project is also deployed and reachable online, split across two managed platforms rather than run on a single host the way docker-compose or a VM would run it. The frontend is deployed on Vercel, which builds the React app directly from the connected repository on every push: https://fyp-project-sand.vercel.app/. The backend is deployed on Railway, which builds and runs the same Dockerfile used by the local Docker Compose setup, so the running container is the same artifact either way: https://fypproject-production-7f42.up.railway.app/, with its health check reachable at https://fypproject-production-7f42.up.railway.app/api/health. This split-platform deployment needs one piece of configuration the single-host Docker Compose setup does not: the backend's CORS origin (env.clientOrigin in App.mjs) must be set to the Vercel frontend's URL, since the two halves no longer share an origin the way nginx's reverse proxy gives them locally. Everything else the backend depends on — MongoDB Atlas, Cloudinary, and the third-party AI providers — is reached identically regardless of which of the three run modes in this appendix is being used.